

COMPLETE INTERVIEW PREPARATION HANDBOOK

The Systems Engineer's Interview Regime

*A comprehensive study system for the
Front Office Technical Associate role at
D. E. Shaw & Co.*

Target Role: Front Office Technical Associate

Specialization: Systems Programming & Networking

Preparation Duration: 8 Weeks (Intensive)

Focus Project: UnMTP Hybrid File Transfer System

Last Updated: July 2026

This handbook is designed for candidates with a systems programming background targeting production engineering, infrastructure engineering, and trading systems support roles. It prioritizes depth in computer networks, operating systems, Linux systems administration, and Python systems programming over web development frameworks.

Table of Contents

8-Week Intensive Schedule

Chapter 1: Computer Networks

- 1.1 Application Layer
- 1.2 Transport Layer
- 1.3 Network Layer
- 1.4 Link & Physical Layers
- 1.5 Network Tools & Troubleshooting

Chapter 2: Operating Systems

- 2.1 Processes & Threads
- 2.2 Memory Management
- 2.3 Synchronization & Deadlocks
- 2.4 I/O Systems & Zero-Copy

Chapter 3: Python for Systems Programming

Chapter 4: Linux Systems

Chapter 5: Debugging & Troubleshooting

Chapter 6: Data Structures & Algorithms

Chapter 7: Object-Oriented Programming

Chapter 8: SQL

Chapter 9: Git

Chapter 10: Computer Architecture

Chapter 11: System Design Basics

Chapter 12: Concurrency

Chapter 13: File Systems

Chapter 14: Networking Tools

Chapter 15: Security Basics

Chapter 16: Behavioral Interview

Chapter 17: Resume Discussion

Chapter 18: Project Discussion (100+ Questions)

Architecture Questions
Networking Questions
Operating Systems Questions
Performance Questions
Security Questions
Python Questions
Debugging Questions
Scalability Questions
Design Decision Questions
Tradeoff Questions
Future Improvement Questions
Failure Case Questions
Testing Questions
Cross-Platform Questions
Optimization Questions
System Call Questions
Binary Protocol Questions

8-Week Intensive Preparation Schedule

Total Hours: ~320 hours **Daily Commitment:** 6-8 hours **Weekends:** 10-12 hours

SCHEDULE PHILOSOPHY

This schedule is designed around your UnMTP project profile. The heaviest emphasis is on Computer Networks (30%), Operating Systems (25%), and Python Systems Programming (15%) because these directly map to the D. E. Shaw Front Office Technical Associate role requirements. DSA is present but not dominant; web frameworks are intentionally excluded.

Weekly Breakdown

Table 1 8-Week Topic Allocation

Week	Primary Focus	Secondary Focus	Practice	Hours
Week 1	Computer Networks (Layers 1-3)	Wireshark Labs	Socket exercises	40
Week 2	Computer Networks (Layer 4 + Tools)	Binary Protocols	TCP/UDP coding	40
Week 3	Operating Systems Core	Linux Systems	Process exercises	40
Week 4	Operating Systems Advanced	Zero-Copy Deep Dive	Memory mapping labs	40
Week 5	Python Systems Programming	Concurrency	Build mini-tools	40
Week 6	DSA Intensive	File Systems	50 problems/week	40
Week 7	System Design + Security	SQL + Git	Mock interviews	40
Week 8	Project Discussion (100+ Qs)	Behavioral Prep	Full mock rounds	40

Daily Time Distribution

Table 2 Recommended Daily Study Block (8 hours)

Time Block	Duration	Activity
Morning Block 1	2 hours	Theory reading / Video lectures
Morning Block 2	2 hours	Hands-on practice / Coding exercises
Afternoon Block 1	2 hours	Problem solving / DSA or System exercises
Afternoon Block 2	1.5 hours	Project review / Interview question practice
Evening	0.5 hour	Revision / Flashcards / Notes consolidation

Priority Matrix

Table 3 Subject Priority and Time Allocation

Priority	Subject	% Time	Est. Hours	Weeks
P0 (Critical)	Computer Networks	22%	70	1-2
P0 (Critical)	Operating Systems	20%	64	3-4
P0 (Critical)	Python Systems Programming	12%	38	5
P1 (High)	Linux Systems	10%	32	3-5
P1 (High)	Debugging & Troubleshooting	6%	19	5
P1 (High)	Data Structures & Algorithms	10%	32	6
P2 (Medium)	Project Discussion (100+ Qs)	8%	26	8
P2 (Medium)	System Design Basics	4%	13	7
P2 (Medium)	Concurrency	3%	10	5
P2 (Medium)	File Systems	2%	6	6
P3 (Foundation)	OOP, SQL, Git, Arch, Security	3%	10	7

Week-by-Week Detailed Plan

Week 1: Computer Networks - Foundation

Day	Morning (Theory)	Afternoon (Practice)
Day 1	OSI and TCP/IP models, encapsulation	Wireshark setup, capture first packets
Day 2	Physical Layer: signals, media, NICs	Exercise: examine interface stats (ifconfig/ip)
Day 3	Data Link Layer: frames, MAC, Ethernet, switches	ARP table examination, MAC address lookup
Day 4	Network Layer: IP, packets, routers, subnetting	IP subnetting exercises, routing table analysis
Day 5	ICMP, ping, traceroute internals	Build ping clone in Python (raw sockets)
Weekend	Review all Week 1 topics	Lab: Packet capture analysis with Wireshark

Week 2: Computer Networks - Transport & Application

Day	Morning (Theory)	Afternoon (Practice)
Day 8	TCP deep dive: handshake, termination, states	Socket programming: echo server/client
Day 9	TCP: sliding window, flow control, congestion	Build custom protocol header with struct
Day 10	UDP: datagrams, use cases, comparison	UDP echo server, compare with TCP
Day 11	DNS: hierarchy, records, resolution process	dig/nslookup exercises, DNS packet capture
Day 12	HTTP/HTTPS: methods, headers, TLS handshake	Build simple HTTP client with raw sockets
Weekend	Network troubleshooting deep dive	Full network stack troubleshooting lab

Week 3: Operating Systems - Core Concepts

Day	Morning (Theory)	Afternoon (Practice)
Day 15	OS structure, kernel vs user mode, system calls	strace exercises on your Python scripts

Day 16	Processes: PCB, states, fork/exec, IPC	Write fork/exec examples in C and Python
Day 17	Threads: user vs kernel, models, pthreads	Python threading examples, GIL exploration
Day 18	Scheduling: algorithms, metrics, CFS	Examine Linux scheduler, nice values
Day 19	Synchronization: mutex, semaphore, monitor	Producer-consumer in Python (threads)
Weekend	Deadlocks: conditions, prevention, avoidance	Dining philosophers simulation

Week 4: Operating Systems - Memory & I/O

Day	Morning (Theory)	Afternoon (Practice)
Day 22	Virtual memory: paging, TLB, page tables	Examine /proc/[pid]/maps
Day 23	Page replacement algorithms	Simulate LRU, FIFO, Clock algorithms
Day 24	Memory mapping (mmap), shared memory	Python mmap example, shared buffer
Day 25	Zero-copy: sendfile(), splice(), DMA	Deep dive: trace sendfile() syscalls
Day 26	File descriptors, I/O multiplexing (select/poll/epoll)	Build select()-based multi-client server
Weekend	Full OS review	Build mini file transfer with zero-copy

Week 5: Python Systems Programming + Concurrency

Day	Morning (Theory)	Afternoon (Practice)
Day 29	socket module: all APIs, options, patterns	Rewrite UnMTP socket layer from scratch
Day 30	struct module: packing, unpacking, endianness	Design new binary protocol headers
Day 31	os, pathlib, sys: filesystem and system APIs	File traversal tools, path manipulation
Day 32	threading, multiprocessing, GIL, pools	Parallel file transfer experiment
Day 33	selectors, asyncio: event-driven I/O	Async echo server, benchmark vs threaded
Weekend	Generators, context managers, decorators	Build instrumentation decorators

Week 6: DSA Intensive + File Systems

Day	Morning (Theory)	Afternoon (Practice)
Day 36	Arrays: prefix sum, sliding window, two pointer	10 LeetCode problems (Easy/Med)
Day 37	Strings: KMP, rolling hash, pattern matching	10 problems + implement KMP

Day 38	Hashing: hash maps, sets, collision handling	10 problems + hash table implementation
Day 39	Trees: BST, traversals, AVL basics	10 tree problems
Day 40	Graphs: BFS, DFS, Dijkstra basics	10 graph problems
Weekend	File systems: FAT, ext4, inodes, journaling	Examine ext4 with debugfs

Week 7: System Design + Supporting Topics

Day	Morning (Theory)	Afternoon (Practice)
Day 43	System design: load balancing, caching, CDNs	Design file transfer system (your project)
Day 44	Database design: sharding, replication, CAP	Design database for file metadata
Day 45	SQL: joins, indexes, transactions, optimization	20 SQL problems (LeetCode/HackerRank)
Day 46	Security: TLS/SSL, encryption, auth basics	Add TLS to your socket project
Day 47	Git internals: objects, refs, plumbing	Practice rebase, reflog, bisect
Weekend	OOP design principles, SOLID	Refactor project with SOLID principles

Week 8: Project Discussion + Mock Interviews

Day	Morning (Theory)	Afternoon (Practice)
Day 50	Project deep dive: architecture questions	Write out 20 detailed answers
Day 51	Project deep dive: networking questions	Write out 20 detailed answers
Day 52	Project deep dive: OS/performance questions	Write out 20 detailed answers
Day 53	Project deep dive: security/debugging questions	Write out 20 detailed answers
Day 54	Behavioral interview preparation	Mock behavioral interview
Weekend	Full mock technical interviews	Timed mock with feedback

CRITICAL SUCCESS FACTORS

- Do not skip hands-on exercises. Theory alone will not get you through D. E. Shaw interviews.
- Every day must include at least 1 hour of active coding or system experimentation.
- Week 8 is non-negotiable: you must complete all 100+ project questions with written answers.
- Sleep 7+ hours. Cramming degrades interview performance more than missing a few topics.
- Track your progress daily. If you fall behind, reduce scope (go deeper on fewer topics) rather than rushing.

01

Computer Networks

Difficulty: High **Importance:** Critical (P0) **Estimated Hours:** 70 **Weeks:** 1-2

WHY THIS MATTERS FOR D. E. SHAW

Front Office Technical Associates work directly with trading infrastructure, market data feeds, and low-latency systems. Every trading system depends on network reliability, throughput, and latency. Your interviewer will expect deep comfort with TCP internals because trading systems must handle market data over TCP/UDP multicast. Understanding congestion control matters because microbursts in market data can overwhelm naive receivers. Understanding socket buffering matters because buffer bloat adds latency that directly impacts P&L. The UnMTP project demonstrates practical networking knowledge, but D. E. Shaw will push far deeper into the protocol stack.

Chapter Overview

This chapter covers the complete network stack from physical transmission to application-layer protocols. Study every section thoroughly. There is no "optional" material in this chapter. D. E. Shaw interviewers are known to ask candidates to trace a packet from application `send()` through every layer to the wire, explaining every transformation at each step.

1.1 Network Fundamentals and Models

OSI Seven-Layer Model

Know every layer by name, number, purpose, and the data unit at each layer. Be able to recite this in your sleep.

Table 1.1 OSI Model - Complete Reference

Layer	Name	Data Unit	Key Devices/Protocols	Function
-------	------	-----------	-----------------------	----------

7	Application	Data	HTTP, FTP, SMTP, DNS	User interface and network services
6	Presentation	Data	SSL/TLS, ASCII, JPEG, MPEG	Translation, encryption, compression
5	Session	Data	NetBIOS, RPC, PPTP	Session establishment and management
4	Transport	Segment	TCP, UDP	End-to-end delivery, reliability
3	Network	Packet	IP, ICMP, ARP, Router	Logical addressing and routing
2	Data Link	Frame	Ethernet, MAC, Switch, Bridge	Physical addressing, framing
1	Physical	Bits	Cable, Hub, Repeater, NIC	Bit transmission over media

TCP/IP Five-Layer Model

The practical model. Know how it maps to OSI. The TCP/IP model combines OSI layers 5-7 into a single Application layer.

Table 1.2 TCP/IP Model vs OSI Model

TCP/IP Layer	OSI Equivalent	Protocols	Key Concepts
Application	5, 6, 7	HTTP, DNS, FTP, SSH	User-facing protocols
Transport	4	TCP, UDP	Ports, segments, sockets
Network	3	IP, ICMP, OSPF, BGP	IP addresses, routing
Data Link	2	Ethernet, Wi-Fi, ARP	MAC addresses, frames
Physical	1	Ethernet cable, fiber, radio	Signals, encoding

Encapsulation and Decapsulation

Master this process. You must be able to explain exactly what happens at each layer when data moves down the stack (encapsulation) and up the stack (decapsulation).

ENCAPSULATION PROCESS (DOWN THE STACK)

1. **Application:** User data is generated (HTTP request, DNS query)
2. **Transport:** TCP/UDP header added (source port, dest port, sequence numbers, flags). Unit becomes a *segment*.
3. **Network:** IP header added (source IP, dest IP, TTL, protocol). Unit becomes a *packet*.
4. **Data Link:** Ethernet header added (source MAC, dest MAC, EtherType) and trailer (FCS). Unit becomes a *frame*.
5. **Physical:** Frame converted to electrical/optical/radio signals and transmitted

Key Terminology to Master

- **MTU (Maximum Transmission Unit):** Maximum size of a packet that can be transmitted without fragmentation. Ethernet MTU = 1500 bytes. Jumbo frames = 9000 bytes.
- **MSS (Maximum Segment Size):** Maximum amount of TCP data in a single segment. $MSS = MTU - IP \text{ header } (20) - TCP \text{ header } (20) = 1460 \text{ bytes for Ethernet.}$
- **RTT (Round Trip Time):** Time for a packet to travel to destination and back.
- **Bandwidth-Delay Product:** $BDP = \text{bandwidth} * RTT$. Determines optimal window size.
- **Throughput:** Actual data transfer rate achieved, always less than or equal to bandwidth.

STOP BOUNDARY - NETWORK FUNDAMENTALS

Do NOT study: OSI implementation details in specific operating systems, X.25, Frame Relay, ATM networks, SONET/SDH timing details. Do NOT go deeper into physical layer modulation schemes beyond basic concepts.

1.2 Physical Layer

Concepts to Master

- Types of transmission media: twisted pair (Cat5e/Cat6/Cat6a), coaxial, fiber optic (single-mode vs multi-mode)
- Signal encoding: NRZ, Manchester encoding
- NIC operation: MAC address (48-bit, burned into hardware, first 24 bits = OUI)
- Hubs vs switches: hubs broadcast to all ports, switches learn MAC addresses and forward selectively
- Full-duplex vs half-duplex communication
- Baseband vs broadband transmission

CONNECTION TO UNMTP

Your project assumes a LAN environment. Understanding that your TCP connection traverses Ethernet frames over Wi-Fi (CSMA/CA) or wired Ethernet (CSMA/CD) helps explain real-world performance characteristics. The 8MB buffer size you chose should be compared against the BDP of your typical LAN connection.

1.3 Data Link Layer

Concepts to Master

Framing

How raw bits are organized into frames. Know byte stuffing and bit stuffing techniques.

Error Detection

- **Parity check:** Single bit added to make total 1s even or odd. Catches single-bit errors only.
- **Checksum:** Sum of all bytes transmitted. Used in IP header (but NOT in Ethernet).
- **CRC (Cyclic Redundancy Check):** Polynomial division. Ethernet uses CRC-32. Much stronger than checksum.

MAC Addressing

- 48-bit address, written as six hex octets (e.g., 00:1A:2B:3C:4D:5E)
- First 24 bits: OUI (Organizationally Unique Identifier) - identifies manufacturer
- Broadcast MAC: FF:FF:FF:FF:FF:FF

ARP (Address Resolution Protocol)

Critical protocol. Maps IP addresses to MAC addresses on local network.

ARP OPERATION

1. Host A wants to send to 192.168.1.5 but needs its MAC address
2. Host A broadcasts ARP Request: "Who has 192.168.1.5? Tell 192.168.1.1"
3. Host with 192.168.1.5 replies with ARP Reply containing its MAC
4. Host A caches this mapping in ARP table (view with `arp -a`)
5. ARP entries age out (typically 4 minutes on Linux)

Ethernet Frame Format

```
| Preamble (7) | SFD (1) | Dest MAC (6) | Src MAC (6) | EtherType (2) | Payload (46-1500) | FCS (4) |
```

Switches vs Bridges vs Hubs

Learns MACs, filters frames

Table 1.3 Layer 2 Device Comparison

Device	Layer	Operation	Collision Domain	Broadcast Domain
Hub	Physical (1)	Electrical repeater, broadcasts all	One	One

Bridge	Data Link (2)	Per port		One
Switch	Data Link (2)	Multi-port bridge, hardware forwarding	Per port	One (without VLANs)

VLANs (Virtual LANs)

- Logically segment a switch into multiple broadcast domains
- VLAN tag: 4-byte IEEE 802.1Q header inserted after source MAC
- VLAN ID: 12 bits, supporting 4094 VLANs (0 and 4095 reserved)
- Trunk ports carry traffic for multiple VLANs

STOP BOUNDARY - DATA LINK LAYER

Do NOT study: Spanning Tree Protocol internals (know what it does and why), MPLS, PPP protocol details, HDLC, SLIP. Know what STP prevents (loops) but not BPDU message formats.

1.4 Network Layer

Concepts to Master

IPv4

```
IPv4 Header (20-60 bytes):
| Version (4) | IHL (4) | TOS (8) | Total Length (16) |
| Identification (16) | Flags (3) | Fragment Offset (13) |
| TTL (8) | Protocol (8) | Header Checksum (16) |
| Source IP Address (32) |
| Destination IP Address (32) |
| Options (variable) | Padding |
```

- **Version:** 4 for IPv4
- **IHL (Internet Header Length):** Number of 32-bit words in header. Minimum 5 (20 bytes).
- **TOS (Type of Service):** Now DSCP (Differentiated Services Code Point) for QoS
- **TTL (Time To Live):** Decrement at each router. Prevents infinite loops. Starts at 64 (Linux) or 128 (Windows).
- **Protocol:** 6=TCP, 17=UDP, 1=ICMP, 2=IGMP
- **Flags:** DF (Don't Fragment), MF (More Fragments)

IP Addressing and Subnetting

You must be able to solve subnetting problems quickly.

Table 1.4 IPv4 Address Classes

Class	First Bits	Range	Default Mask	Use
A	0	1.0.0.0 - 126.255.255.255	/8	Large networks
B	10	128.0.0.0 - 191.255.255.255	/16	Medium networks
C	110	192.0.0.0 - 223.255.255.255	/24	Small networks
D	1110	224.0.0.0 - 239.255.255.255	N/A	Multicast
E	1111	240.0.0.0 - 255.255.255.255	N/A	Reserved

Private IP Ranges (RFC 1918)

- 10.0.0.0/8 (10.0.0.0 - 10.255.255.255)
- 172.16.0.0/12 (172.16.0.0 - 172.31.255.255)
- 192.168.0.0/16 (192.168.0.0 - 192.168.255.255)

Special Addresses

- **127.0.0.0/8**: Loopback (127.0.0.1 = localhost)
- **0.0.0.0**: Default route, or "all interfaces"
- **255.255.255.255**: Limited broadcast
- **169.254.0.0/16**: Link-local (APIPA)

CIDR (Classless Inter-Domain Routing)

Understand CIDR notation and be able to calculate network addresses, broadcast addresses, usable host ranges, and number of hosts.

SUBNETTING QUICK REFERENCE

CIDR	Subnet Mask	Hosts	Wildcard
/24	255.255.255.0	254	0.0.0.255
/16	255.255.0.0	65,534	0.0.255.255
/8	255.0.0.0	16,777,214	0.255.255.255
/30	255.255.255.252	2	0.0.0.3

IPv6 (Basic Awareness)

- 128-bit addresses, written as 8 groups of 4 hex digits
- Example: 2001:0db8:85a3::8a2e:0370:7334
- :: compresses consecutive zero groups (can only use once)

- Loopback: ::1
- No broadcast; uses multicast and anycast
- Dual-stack: IPv4 and IPv6 running simultaneously

ICMP (Internet Control Message Protocol)

- **Type 0 / Code 0:** Echo Reply (ping response)
- **Type 8 / Code 0:** Echo Request (ping request)
- **Type 3:** Destination Unreachable (Code 0=net, 1=host, 3=port)
- **Type 11:** Time Exceeded (TTL=0, used by traceroute)
- **Type 5:** Redirect (used by routers)

Routing Basics

- **Static routing:** Manually configured routes
- **Dynamic routing:** Protocols automatically discover paths
 - **IGP (Interior Gateway Protocol):** RIP (distance vector), OSPF (link state), EIGRP
 - **EGP (Exterior Gateway Protocol):** BGP (Border Gateway Protocol)
- **Routing table:** View with `route -n` OR `ip route`
- **Longest prefix match:** Router selects most specific route

NAT (Network Address Translation)

Critical for understanding how your UnMTP client connects across networks.

- **Static NAT:** One-to-one mapping of private to public IP
- **Dynamic NAT:** Pool of public IPs assigned dynamically
- **PAT/NAPT:** Many-to-one mapping using different port numbers (most common)
- NAT modifies IP header (breaks end-to-end principle)
- NAT traversal techniques: UPnP, NAT-PMP, STUN, TURN, ICE

CONNECTION TO UNMTP

Your UnMTP system uses port 5001 on a private LAN. If you ever needed to transfer across NAT (e.g., different networks), you would need NAT traversal. Your gateway auto-detection finds the router's LAN IP (e.g., 192.168.1.1) - understand that this is the default gateway, the router's interface on your local network.

STOP BOUNDARY - NETWORK LAYER

Do NOT study: OSPF LSA types and SPF algorithm details, BGP path selection algorithm internals, MPLS label distribution, IPsec AH/ESP packet formats in detail, IPv6 extension header chain processing. Know what OSPF and BGP do conceptually but not their packet formats.

1.5 Transport Layer

CRITICAL IMPORTANCE

This is the single most important section in the entire handbook. D. E. Shaw interviewers will spend the most time here. Your UnMTP project is built directly on TCP sockets. Every design decision in your project (8MB buffer, 1MB chunk size, 60s timeout, SO_REUSEADDR) relates to transport layer concepts. You must be able to explain each of these decisions with precise technical reasoning.

TCP (Transmission Control Protocol)

TCP Header Format (MUST MEMORIZE)

```
TCP Header (20-60 bytes):
  0          1          2          3
  0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Source Port           |           Destination Port       |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Sequence Number       |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Acknowledgment Number |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Data |           |U|A|P|R|S|F|           |
| Offset| Reserved  |R|C|S|S|Y|I|           Window           |
|       |           |G|K|H|T|N|N|           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Checksum           |           Urgent Pointer         |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Options (optional)           |Pad|
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
```

Every Field Explained

Table 1.5 TCP Header Fields

Field	Size	Purpose
-------	------	---------

Source Port	16 bits	Sender's port number
Destination Port	16 bits	Receiver's port number
Sequence Number	32 bits	Byte sequence number of first data byte in segment (or ISN+1 for SYN)
Acknowledgment Number	32 bits	Next expected byte (valid only if ACK flag set)
Data Offset	4 bits	TCP header size in 32-bit words. Min 5 (20 bytes).
Flags (6)	6 bits	URG, ACK, PSH, RST, SYN, FIN
Window	16 bits	Receive window size (flow control)
Checksum	16 bits	Error detection (header + data + pseudo-header)
Urgent Pointer	16 bits	Offset to urgent data (if URG set)
Options	Variable	MSS, window scale, timestamps, SACK

TCP Flags - Know Each One

Table 1.6 TCP Flags Reference

Flag	Name	When Set	Interview Relevance
SYN	Synchronize	Connection establishment (first packet)	Three-way handshake
ACK	Acknowledgment	Acknowledging received data	Every packet after SYN
FIN	Finish	Graceful connection termination	Four-way termination
RST	Reset	Abort connection immediately	Port closed, error recovery
PSH	Push	Send data immediately to application	Interactive applications
URG	Urgent	Urgent data present	Rarely used (telnet)

Three-Way Handshake (SYN, SYN-ACK, ACK)

You must be able to draw this from memory and explain every field value.

THREE-WAY HANDSHAKE - STEP BY STEP

Step 1 - SYN: Client sends SYN to server. Seq = ISN_c (random 32-bit), Ack = 0, Flags = SYN.

Step 2 - SYN-ACK: Server sends SYN-ACK. Seq = ISN_s (random), Ack = ISN_c + 1, Flags = SYN+ACK.

Step 3 - ACK: Client sends ACK. Seq = ISN_c + 1, Ack = ISN_s + 1, Flags = ACK. Connection established.

Key Points:

- Both sides choose independent initial sequence numbers (ISN) for security (prevent TCP hijacking)
- SYN consumes one sequence number (hence ISN + 1)
- The third packet can carry data (though typically doesn't)
- Half-open connection: SYN received but SYN-ACK not yet acknowledged

Four-Way Termination (FIN, FIN-ACK, FIN, FIN-ACK)

FOUR-WAY TERMINATION - STEP BY STEP

Step 1: Client sends FIN. Seq = next expected, Flags = FIN+ACK.

Step 2: Server sends ACK for FIN. Ack = client's FIN seq + 1. Connection half-closed.

Step 3: Server sends its own FIN when ready. Flags = FIN+ACK.

Step 4: Client sends ACK. Enters TIME_WAIT state for 2*MSL.

Key Points:

- TCP is full-duplex; each direction must be closed independently
- FIN means "I have no more data to send" but can still receive
- ACK of FIN is the last packet in that direction
- Simultaneous close: both sides send FIN at same time (uncommon)

TCP Connection States (MUST KNOW ALL)

Table 1.7 TCP Connection States

State	Description	Typical Duration
CLOSED	No connection	N/A
LISTEN	Server waiting for connection	Entire session
SYN_SENT	Client sent SYN, waiting for SYN-ACK	~1 RTT

SYN_RECEIVED	Server received SYN, sent SYN-ACK, waiting ACK	~1 RTT
ESTABLISHED	Connection active, data transfer	Entire session
FIN_WAIT_1	Sent FIN, waiting ACK or peer's FIN	Brief
FIN_WAIT_2	Received ACK for FIN, waiting peer's FIN	Can be long
CLOSE_WAIT	Received FIN, sent ACK, waiting app to close	Application-dependent
CLOSING	Simultaneous close, waiting ACK	Brief
LAST_ACK	Sent FIN after CLOSE_WAIT, waiting final ACK	~1 RTT
TIME_WAIT	Received final ACK, waiting 2*MSL	2-4 minutes

TIME_WAIT State - Deep Understanding Required

WHY TIME_WAIT EXISTS (2 MINUTES ON LINUX)

1. **Reliable connection termination:** If the final ACK is lost, the server retransmits its FIN. The client in TIME_WAIT can re-send the ACK.
2. **Prevent old segments from new connections:** Without TIME_WAIT, a new connection with the same 4-tuple could receive delayed segments from the old connection.

Duration: 2 * MSL (Maximum Segment Lifetime). MSL = 30 seconds on Linux, so TIME_WAIT = 60 seconds. Windows uses 2 minutes.

Problem: High-churn servers can exhaust ephemeral ports due to many sockets in TIME_WAIT.

Solutions: SO_REUSEADDR (your project uses this), tcp_tw_reuse (Linux), tcp_tw_recycle (deprecated, dangerous).

CONNECTION TO UNMTP

Your UnMTP host uses `SO_REUSEADDR` precisely to avoid "Address already in use" errors when restarting the server quickly. Without this option, the socket would remain in TIME_WAIT for 60 seconds after the previous transfer, preventing immediate rebind. Be prepared to explain this in detail.

CLOSE_WAIT State - Debugging Implication

Many sockets in CLOSE_WAIT indicates the application received a FIN but never called close(). This is an **application bug**, not a network issue. The peer has closed its end, but your application is not closing its socket. Check with: `netstat -an | grep CLOSE_WAIT`

Sliding Window Protocol

TCP uses a sliding window for both reliability and flow control. The window allows the sender to transmit multiple segments before receiving acknowledgment.

HOW SLIDING WINDOW WORKS

1. Sender maintains a window of bytes it can send without ACK
2. As ACKs arrive, the window slides forward
3. Window size is determined by the receiver's advertised window (flow control)
4. Sender also respects congestion window (congestion control)
5. Effective window = $\min(\text{advertised window}, \text{congestion window})$

TCP Flow Control

Prevents fast sender from overwhelming slow receiver.

- Receiver advertises window size in every ACK (Window field in TCP header)
- Window size can be 0 (zero-window): sender stops transmitting
- Zero-window probe: sender periodically sends 1-byte segments to check if window reopened
- Window scaling (RFC 1323): allows window sizes $> 65,535$ bytes (necessary for high-BDP networks)

TCP Congestion Control (CRITICAL)

Four algorithms. Know each one precisely.

Table 1.8 TCP Congestion Control Algorithms

Phase	Algorithm	Behavior	Trigger
Start	Slow Start	cwnd doubles every RTT (exponential growth)	New connection or timeout
Growth	Congestion Avoidance	cwnd increases by 1 MSS per RTT (linear)	cwnd reaches ssthresh
Fast Recovery	Fast Retransmit	Retransmit on 3 duplicate ACKs (not timeout)	3 dup ACKs
Recovery	Fast Recovery	cwnd = ssthresh + 3 MSS, then linear	After fast retransmit

CONGESTION CONTROL STATE MACHINE

1. **Initial:** cwnd = 1-10 MSS (Linux default initcwnd = 10), ssthresh = infinite
2. **Slow Start:** cwnd doubles each RTT. Continues until cwnd >= ssthresh or packet loss.
3. **On timeout:** ssthresh = cwnd/2, cwnd = 1 MSS. Back to slow start.
4. **On 3 dup ACKs:** ssthresh = cwnd/2, cwnd = ssthresh (or ssthresh+3 in fast recovery).
5. **Congestion Avoidance:** cwnd += MSS * MSS / cwnd per ACK (approx 1 MSS per RTT).

TCP Options (Know the Important Ones)

Table 1.9 Important TCP Options

Option	Kind	Purpose
MSS	2	Maximum Segment Size (sent in SYN)
Window Scale	3	Multiply window by 2^{scale} (for BDP > 64KB)
SACK	5	Selective Acknowledgment (ack out-of-order blocks)
Timestamp	8	RTT measurement, PAWS protection
NOP	1	Padding to align options on 4-byte boundary

Retransmission

- **RTO (Retransmission Timeout):** Calculated from RTT measurements using Jacobson/Karels algorithm
- **Exponential backoff:** RTO doubles after each consecutive timeout (1x, 2x, 4x, 8x... capped at 64x)
- **Fast Retransmit:** Retransmit after 3 duplicate ACKs without waiting for RTO
- **SACK-based recovery:** Retransmit only missing segments (more efficient)

Keep-Alive

- TCP keep-alive probes sent after period of inactivity (default 2 hours on Linux)
- 9 probes at 75-second intervals before declaring connection dead
- Can be configured with `SO_KEEPALIVE`, `TCP_KEEPIDLE`, `TCP_KEEPINTVL`, `TCP_KEEPCNT`
- Your UnMTP uses a 60-second socket timeout instead of TCP keepalive

Nagle's Algorithm and TCP_NODELAY

- **Nagle's Algorithm:** Delays sending small segments until ACK of previous data arrives. Reduces packet count but adds latency.
- **TCP_NODELAY:** Disables Nagle's. Send data immediately. Essential for low-latency applications (trading).
- **Interaction with Delayed ACKs:** Nagle + delayed ACK = terrible performance (200ms stalls). Either disable Nagle or disable delayed ACKs.

TCP vs UDP - Complete Comparison

Table 1.10 TCP vs UDP Comparison

Feature	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed delivery, ordered	Best-effort, unordered
Flow Control	Yes (sliding window)	No
Congestion Control	Yes	No
Header Size	20-60 bytes	8 bytes
Overhead	High (ACKs, state)	Minimal
Use Cases	File transfer, HTTP, SSH	DNS, video streaming, gaming, market data
Broadcast/Multicast	No	Yes

UDP (User Datagram Protocol)

UDP Header Format

```
UDP Header (8 bytes):
| Source Port (16) | Destination Port (16) |
| Length (16)    | Checksum (16)    |
```

- **Source Port:** Optional (0 = not used)
- **Length:** UDP header + data in bytes
- **Checksum:** Optional in IPv4, mandatory in IPv6 (covers pseudo-header)

When to Use UDP Over TCP

- Real-time applications where latency matters more than reliability (VoIP, video conferencing)
- DNS queries (single request-response, timeout is simpler than TCP overhead)
- Multiplexed streaming (QUIC, HTTP/3)
- Financial market data feeds (firms often use UDP multicast for price ticks)
- When application can handle reliability itself

CONNECTION TO UNMTP - WHY TCP NOT UDP

Your UnMTP uses TCP because file transfer requires reliability. Every byte must arrive correctly and in order. UDP would require implementing retransmission, ordering, and congestion control at application layer - essentially rebuilding TCP. However, for a future iteration adding market data-like streaming (progress updates), a parallel UDP channel could be considered. This shows systems thinking.

STOP BOUNDARY - TRANSPORT LAYER

Do NOT study: TCP Vegas/Veno/Illinois algorithm internals, TCP offload engines (TSO/LRO) beyond awareness, SCTP protocol details, DCCP, MPTCP. Do NOT memorize exact RFC numbers. Know the concepts of TCP Fast Open (TFO) but not packet format. Know what QUIC/HTTP3 does conceptually but not implementation.

1.6 Application Layer

DNS (Domain Name System)

How DNS Resolution Works

DNS RESOLUTION PROCESS

1. Check browser/OS DNS cache
2. Query configured DNS resolver (typically ISP or 8.8.8.8)
3. Resolver checks its cache; if miss, performs recursive resolution:
4. Query root server (.) - gets TLD server address
5. Query TLD server (.com) - gets authoritative server address
6. Query authoritative server (example.com) - gets final IP
7. Response cached at each level with TTL

DNS Record Types

Table 1.11 Common DNS Record Types

Type	Purpose	Example
------	---------	---------

A	IPv4 address	example.com → 93.184.216.34	
AAAA	IPv6 address	example.com 2606:2800:220:1:248:1893:25c8:1946	→
CNAME	Alias to another domain	www.example.com → example.com	
MX	Mail server	example.com → 10 mail.example.com	
NS	Authoritative name server	example.com → ns1.example.com	
TXT	Text record (SPF, DKIM)	v=spf1 include:_spf.google.com ~all	
PTR	Reverse DNS (IP to name)	34.216.184.93.in-addr.arpa → example.com	
SOA	Zone authority info	Primary NS, admin email, serial, refresh times	

DNS Transport

- Typically UDP port 53 for queries (faster, stateless)
- TCP port 53 for zone transfers and responses > 512 bytes
- EDNS0 extends UDP to larger sizes
- DNS over HTTPS (DoH) and DNS over TLS (DoT) for privacy

HTTP/HTTPS

HTTP/1.1 vs HTTP/2 vs HTTP/3

Table 1.12 HTTP Version Comparison

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	UDP (QUIC)
Multiplexing	No (pipelining limited)	Yes (streams)	Yes (streams)
Header Compression	No	HPACK	QPACK
Server Push	No	Yes	Yes
Connection Setup	1 RTT + TLS	1 RTT + TLS	0-1 RTT
Head-of-Line Blocking	Yes	Yes (TCP level)	No

TLS Handshake (1.2 vs 1.3)

TLS 1.2 HANDSHAKE (2 RTT)

1. Client Hello: supported cipher suites, random nonce, key share (optional)
2. Server Hello: chosen cipher, certificate, Server Key Exchange, Server Hello Done
3. Client Key Exchange: pre-master secret encrypted with server public key, Change Cipher Spec
4. Server Change Cipher Spec, encrypted handshake finished

TLS 1.3: 1-RTT handshake (0-RTT with resumption). Simplified, removed RSA key exchange (only DHE), fewer cipher suites.

HTTP Methods and Status Codes

Know all common methods: GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS. Know status code classes: 1xx (info), 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error). Know specific common codes: 200, 201, 301, 302, 304, 400, 401, 403, 404, 500, 502, 503.

DHCP (Dynamic Host Configuration Protocol)

DORA PROCESS

1. **Discover:** Client broadcasts DHCPDISCOVER (src 0.0.0.0, dest 255.255.255.255)
2. **Offer:** Server responds with DHCPOFFER containing IP, subnet, gateway, DNS, lease time
3. **Request:** Client broadcasts DHCPREQUEST (accepting offer)
4. **Acknowledge:** Server sends DHCPACK

Ports: UDP 67 (server), UDP 68 (client)

SSH (Secure Shell)

- Port 22, TCP
- Uses public-key cryptography for authentication
- Transport layer: encrypts all traffic
- Key exchange: Diffie-Hellman or elliptic-curve DH
- Host keys stored in ~/.ssh/known_hosts
- Authentication: password, public key, host-based, keyboard-interactive

1.7 Network Tools and Troubleshooting

Essential Tools - Practice Each One

Table 1.13 Network Troubleshooting Tools

Tool	Purpose	Key Options	When to Use
------	---------	-------------	-------------

ping	Test reachability (ICMP echo)	-c count, -i interval, -s size	Basic connectivity
tracert	Trace route path	-I (ICMP), -T (TCP), -U (UDP)	Routing problems
netstat	Network connections and statistics	-tuln (listening TCP/UDP), -anp	Check ports, connections
ss	Socket statistics (faster than netstat)	-tlnp, -s (summary)	Socket inspection
tcpdump	Packet capture	-i interface, -nn, -w file, port 5001	Traffic analysis
Wireshark	GUI packet analyzer	Filters: tcp.port==5001, ip.addr==x.x.x.x	Deep packet inspection
nc/nmap	Port scan, netcat	-zv (scan), -l (listen), -p (port)	Port testing
curl	HTTP client	-v, -I, -H, -X, --trace	HTTP debugging
dig	DNS lookup	+trace, @server, ANY	DNS troubleshooting
lsof	List open files (includes sockets)	-i :5001, -p PID	Find process using port
iptables/nftables	Firewall rules	-L (list), -A (append), -D (delete)	Firewall debugging

Troubleshooting Methodology

LAYERED TROUBLESHOOTING APPROACH

1. **Physical:** Is the cable connected? Interface up? `ip link`, `ethtool`
2. **Data Link:** MAC address correct? ARP resolved? `arp -a`
3. **Network:** IP configured? Routing correct? `ip addr`, `ip route`, `ping`
4. **Transport:** Port listening? Firewall blocking? `ss -tlnp`, `iptables -L`
5. **Application:** Service running? Correct protocol? Logs, `curl`, `telnet`

Resources - Computer Networks

Books

- **Computer Networking: A Top-Down Approach** by Kurose & Ross (7th Ed) - THE primary textbook. Read chapters 1-5 thoroughly. Do the Wireshark labs.
- **TCP/IP Illustrated, Volume 1** by W. Richard Stevens - Reference for TCP internals. Read chapters 17-24 for TCP deep dive.
- **Understanding Linux Network Internals** by Christian Benvenuti - For kernel-level networking after you master basics.

Online Courses

- **Stanford CS144: Computer Network** (YouTube) - Excellent lectures with labs
- **CMU 15-441/641: Computer Networks** - Lecture notes available online
- **MIT 6.829: Computer Networks** - Graduate level, for advanced topics

Documentation

- Beej's Guide to Network Programming (beej.us/guide/bgnet) - Best socket programming guide
- Linux man pages: `socket(2)`, `tcp(7)`, `ip(7)`, `sendfile(2)`
- RFC 793 (TCP), RFC 768 (UDP), RFC 2616 (HTTP/1.1) - Skim for key sections

Interactive Practice

- Wireshark sample captures (wiki.wireshark.org/SampleCaptures)
- Python socket programming exercises
- Build a mini TCP stack (CS144 lab approach)

Practice Exercises

1. Write a TCP echo server and client in Python from scratch (no frameworks)
2. Write a UDP echo server and client
3. Capture a TCP three-way handshake in Wireshark and identify all fields
4. Build a simple HTTP/1.0 client using only socket and string manipulation
5. Implement a basic port scanner using Python sockets
6. Write a script that performs DNS resolution using UDP sockets (no `socket.gethostbyname`)
7. Experiment with TCP window sizes and measure throughput impact
8. Use `tcpdump` to capture UnMTP traffic and analyze the packet structure
9. Implement a reliable transfer protocol over UDP (stop-and-wait)
10. Experiment with `SO_SNDBUF` and `SO_RCVBUF` values, measure transfer times

Expected Interview Questions - Computer Networks

Q1: Explain the TCP three-way handshake in detail. What is in each packet?

See Section 1.5 above. Be specific: SYN has Seq=ISN_c, Ack=0, Flags=SYN. SYN-ACK has Seq=ISN_s, Ack=ISN_c+1, Flags=SYN+ACK. ACK has Seq=ISN_c+1, Ack=ISN_s+1, Flags=ACK.

Q2: Why is the initial sequence number (ISN) random and not 0?

Security: prevents TCP sequence number prediction attacks (connection hijacking). If ISN were predictable, an attacker could inject packets into an existing connection. RFC 793 requires ISN to change every 4 microseconds.

Q3: What happens if the final ACK in the four-way termination is lost?

The server (LAST_ACK state) does not receive ACK, so it retransmits its FIN after a timeout. The client is in TIME_WAIT and will re-send the ACK upon receiving the retransmitted FIN. This is exactly why TIME_WAIT exists.

Q4: Explain TCP congestion control. What happens on timeout vs 3 duplicate ACKs?

On timeout: ssthresh = cwnd/2, cwnd = 1 MSS, enter slow start. On 3 dup ACKs: ssthresh = cwnd/2, cwnd = ssthresh (or ssthresh + 3 in fast recovery), enter congestion avoidance. Timeout is treated as more severe congestion signal.

Q5: What is the difference between flow control and congestion control?

Flow control: prevents sender from overwhelming receiver (receiver's advertised window). Congestion control: prevents sender from overwhelming network (congestion window, inferred from packet loss/ECN). Effective window = min(flow window, congestion window).

Q6: Why did you choose TCP over UDP for your file transfer project?

File transfer requires reliability and ordering. TCP provides both natively. Using UDP would require implementing retransmission, sequencing, and congestion control at application layer - essentially rebuilding TCP. The 8MB buffer and 1MB chunk sizes are optimizations on top of TCP's guarantees.

Q7: What is the Bandwidth-Delay Product and why does it matter?

BDP = bandwidth * RTT. It represents the amount of data that can be "in flight" on the network. To fully utilize a high-BDP link, the receive buffer must be at least as large as the BDP. If the buffer is smaller, the sender will stall waiting for ACKs despite available bandwidth.

Q8: Explain NAT traversal. How would you make your file transfer work across different networks?

NAT maps private IPs to public IPs. For inbound connections, port forwarding or UPnP/NAT-PMP can open a port. Alternative: relay server (TURN) that both clients connect to. STUN helps discover public endpoint. For production, a mediator server with public IP would coordinate transfers.

Q9: What is TCP head-of-line blocking? How does HTTP/2 vs HTTP/3 address it?

In HTTP/1.1, a lost packet blocks all subsequent requests on the same connection. HTTP/2 multiplexes streams over one TCP connection but still suffers from TCP-level head-of-line blocking. HTTP/3 uses QUIC over UDP, so packet loss in one stream doesn't affect others.

Q10: How does TCP calculate retransmission timeout (RTO)?

RTO is calculated from RTT measurements using Jacobson's algorithm: $RTT_{VAR} = (1-\beta) \cdot RTT_{VAR} + \beta \cdot |SRTT - RTT_{sample}|$. $SRTT = (1-\alpha) \cdot SRTT + \alpha \cdot RTT_{sample}$. $RTO = SRTT + 4 \cdot RTT_{VAR}$. Exponential backoff doubles RTO on consecutive timeouts.

Q11: What is the purpose of the TCP checksum if Ethernet already has CRC?

Ethernet CRC only covers the frame on a single link and is recomputed at every hop (router). TCP checksum is end-to-end, covering the entire segment from source to destination host, including a pseudo-header with IP addresses. It detects errors that might occur inside routers.

Q12: Explain the difference between CLOSE_WAIT and TIME_WAIT.

CLOSE_WAIT: local end received FIN from peer, sent ACK, waiting for application to close the socket. Many CLOSE_WAITs indicate application bug. TIME_WAIT: local end sent final ACK after closing, waiting $2 \cdot MSL$ to ensure peer received the ACK and to prevent stale segments.

Q13: What happens when you type a URL into a browser? (Full stack trace)

DNS resolution (cache → resolver → root → TLD → auth), TCP connection (SYN-SYN/ACK-ACK), TLS handshake (ClientHello → ServerHello → key exchange), HTTP request, server processing, HTTP response, rendering. Be ready to expand on any step for 10+ minutes.

Q14: What is MTU and MSS? What happens when a packet exceeds MTU?

MTU = Maximum Transmission Unit (max frame payload, typically 1500 bytes). MSS = MTU - IP header - TCP header (typically 1460 bytes). If packet exceeds MTU and DF bit is set, router sends ICMP "Fragmentation Needed" and drops packet (Path MTU Discovery). If DF is not set, router fragments the packet.

Q15: How does ARP work? What is ARP poisoning?

ARP maps IP to MAC on local network via broadcast request and unicast reply. ARP table caches mappings (`arp -a`). ARP poisoning (spoofing): attacker sends forged ARP replies associating their MAC with another IP, redirecting traffic through attacker. Prevention: static ARP entries, dynamic ARP inspection.

Revision Checklist

- [] Can draw OSI and TCP/IP models from memory with protocols at each layer
- [] Can explain encapsulation process step by step
- [] Can draw TCP three-way handshake with exact sequence/ack numbers
- [] Can draw four-way termination and explain TIME_WAIT
- [] Can list all TCP flags and their purposes
- [] Can explain sliding window, flow control, and congestion control
- [] Can calculate subnet ranges from CIDR notation
- [] Can explain ARP, DNS resolution, and NAT
- [] Can troubleshoot a network problem using the layered approach
- [] Can capture and analyze packets in Wireshark
- [] Can explain every socket option used in your UnMTP project

02

Operating Systems

Difficulty: High **Importance:** Critical (P0) **Estimated Hours:** 64 **Weeks:** 3-4

WHY THIS MATTERS FOR D. E. SHAW

Trading systems run on Linux. Every microsecond matters. Understanding how the kernel schedules processes, manages memory, handles I/O, and optimizes data paths is essential for a Front Office Technical Associate. Your UnMTP project uses `sendfile()` - a kernel system call - and your interviewer will expect you to explain exactly what happens inside the kernel when you call it. D. E. Shaw's infrastructure team needs people who understand the boundary between user space and kernel space.

2.1 OS Structure and System Calls

Kernel vs User Mode

- **User mode:** Applications run here. Restricted access to hardware and memory.
- **Kernel mode:** OS runs here. Full access to all hardware and memory.
- **Mode switch:** Triggered by system calls, interrupts, or exceptions.
- **Protection rings:** Ring 0 (kernel), Ring 3 (user). x86 has 4 rings but Linux uses only 0 and 3.

System Calls

System calls are the interface between user programs and the kernel. Every I/O operation in your UnMTP project ultimately becomes a system call.

Table 2.1 Key System Calls for Your Project

Syscall	Purpose	Used In UnMTP
<code>socket()</code>	Create a socket file descriptor	Yes - both host and pull
<code>bind()</code>	Bind socket to address and port	Yes - host

<code>listen()</code>	Mark socket as passive (accept connections)	Yes - host
<code>accept()</code>	Accept incoming connection	Yes - host
<code>connect()</code>	Initiate connection to remote socket	Yes - pull
<code>sendfile()</code>	Zero-copy file-to-socket transfer	Yes - host (Phase 1)
<code>read()/recv()</code>	Receive data from socket	Yes - pull
<code>write()/send()</code>	Send data through socket	Yes - host (headers)
<code>close()</code>	Close file descriptor	Yes - both
<code>open()</code>	Open a file	Yes - host
<code>mmap()</code>	Map file to memory	No - but know it
<code>select()/poll()/epoll()</code>	I/O multiplexing	No - but know them

System Call Mechanism

HOW A SYSTEM CALL WORKS (X86_64 LINUX)

1. User program loads syscall number into rax register
2. Arguments placed in rdi, rsi, rdx, r10, r8, r9
3. Execute syscall instruction (or int 0x80 legacy)
4. CPU switches to kernel mode, jumps to syscall handler
5. Kernel looks up syscall in `sys_call_table` using rax
6. Kernel function executes with arguments
7. Result placed in rax, return to user mode

Context Switching

When the CPU switches from one process to another:

1. Save state of current process (registers, PC, stack pointer) to PCB
2. Update PCB state (running → ready/waiting)
3. Select next process from ready queue (scheduler)
4. Load state of new process from its PCB
5. Jump to new process's program counter

Cost: Direct (saving/loading registers) + indirect (cache pollution, TLB flush). Can be 1-10 microseconds.

2.2 Processes

Process Concept

- A process is a program in execution: code + data + heap + stack + registers + PCB
- **PCB (Process Control Block)**: Contains PID, state, PC, registers, memory limits, open files, scheduling info
- Processes are isolated: each has its own virtual address space

Process States

Table 2.2 Process States

State	Description	Transition To
New	Being created	Ready
Ready	Waiting to be assigned to CPU	Running
Running	Instructions executing on CPU	Ready, Waiting, Terminated
Waiting/Blocked	Waiting for I/O or event	Ready
Terminated	Execution completed	N/A

fork(), exec(), wait(), exit()

PROCESS CREATION IN UNIX

- **fork()**: Creates exact copy of calling process. Child gets copy-on-write address space. Returns 0 to child, PID to parent, -1 on error.
- **exec()**: Replaces current process image with new program. Does not return on success.
- **wait()**: Parent blocks until child terminates. Collects exit status (prevents zombie).
- **exit()**: Terminates process, returns status to parent.

Zombie and Orphan Processes

- **Zombie**: Process terminated but parent hasn't called wait(). Still has PCB entry. Harmless in small numbers.
- **Orphan**: Parent terminated before child. Child re-parented to init (PID 1) or systemd.
- **Solution**: Parent should always wait() for children, or use SIGCHLD handler with waitpid(-1, &status, WNOHANG).

Inter-Process Communication (IPC)

Table 2.3 IPC Mechanisms

Mechanism		Type	related	Speed	Use Case
Pipes		Half-duplex, processes		Fast	Shell pipelines
Named (FIFO)	Pipes	Half-duplex, processes	unrelated	Fast	Client-server local
Message Queues		Structured messages		Medium	Priority messages
Shared Memory		Fastest, synchronization	needs	Fastest	Large data transfer
Memory-Mapped Files		File-backed shared region		Fast	Persistent shared data
Unix Sockets	Domain	Socket API, local only		Fast	Local client-server
Network Sockets		Cross-machine		Slowest	Network communication
Signals		Async notification		N/A	Events, interrupts

2.3 Threads

Thread Concept

- A thread is a unit of execution within a process
- Threads share code, data, heap, open files, signals
- Each thread has its own: stack, registers, program counter, thread-local storage
- Thread control block (TCB) is smaller than PCB

User-Level vs Kernel-Level Threads

Table 2.4 Thread Models

Model	Managed By		Advantages		Disadvantages
User Threads	Thread library	(green threads)	Fast context switch, any OS		Blocking syscall blocks all threads

Kernel Threads	OS kernel	True parallelism, no blocking issue	Slower dependent	switch, OS
Hybrid (M:N)	Both	Best of both	Complex implementation	

Python's GIL (Global Interpreter Lock)

CONNECTION TO UNMTP - PYTHON THREADING

- Python's CPython implementation has a GIL: only one thread executes Python bytecode at a time
- Threads are still useful for I/O-bound tasks (network I/O, file I/O) because the GIL is released during I/O
- For CPU-bound tasks, use multiprocessing (separate Python interpreters, separate GILs)
- UnMTP is I/O-bound (network transfer), so threading would help if you added concurrent transfers
- Alternative: asyncio with selectors for single-threaded concurrency

2.4 CPU Scheduling

Scheduling Algorithms

Table 2.5 Scheduling Algorithms

Algorithm	Type	Pros	Cons
FCFS	Non-preemptive	Simple	Convoy effect
SJF	Non-preemptive	Optimal avg waiting time	Starvation of long jobs
Shortest Remaining Time First	Preemptive	Responsive	Starvation, context switch overhead
Round Robin	Preemptive	Fair, responsive	High context switch with small quantum
Priority Scheduling	Either	Important jobs first	Starvation (solution: aging)

Multilevel Queue	Preemptive	Different classes	Inflexible
Multilevel Feedback Queue	Preemptive	Adaptive	Complex

Linux CFS (Completely Fair Scheduler)

- Default since Linux 2.6.23
- Uses red-black tree for $O(\log n)$ process selection
- Schedules based on vruntime (virtual runtime), proportional to priority (nice value)
- Target latency: minimum time for all runnable tasks to get one timeslice
- Nice values: -20 (highest) to +19 (lowest), default 0
- Real-time priorities: 1-99 (FIFO and round-robin), always preempt normal tasks

2.5 Synchronization

Critical Section Problem

A critical section is code that accesses shared resources. Need to ensure:

- **Mutual Exclusion:** Only one process in critical section at a time
- **Progress:** If no process is in critical section, selection cannot be postponed indefinitely
- **Bounded Waiting:** Limit on how many times others can enter after a process requests entry

Synchronization Primitives

Mutex (Mutual Exclusion Lock)

- Binary lock: locked or unlocked
- Only the thread that locked it can unlock it (ownership)
- Used for protecting critical sections
- Can cause deadlock if not used carefully

Semaphore

- Counting semaphore: integer value, wait() decrements, signal() increments
- Binary semaphore: like mutex but no ownership requirement
- Can be used for both mutual exclusion and ordering

Condition Variables

- Used with mutex
- wait() releases mutex and blocks until signaled
- signal()/broadcast() wakes one/all waiting threads
- Always check condition in a while loop (spurious wakeups)

Reader-Writer Lock

- Multiple readers simultaneously, or one writer exclusively

- Prefer reader or writer preference variants
- Useful for read-heavy shared data structures

Classic Synchronization Problems

- **Producer-Consumer:** Bounded buffer, producer waits when full, consumer waits when empty
- **Readers-Writers:** Multiple readers or single writer access to shared resource
- **Dining Philosophers:** Deadlock demonstration, solutions: limit philosophers, asymmetric pickup
- **Sleeping Barber:** Coordination between single server and multiple clients

2.6 Deadlocks

Deadlock Conditions (ALL FOUR must hold)

1. **Mutual Exclusion:** Resources are non-shareable
2. **Hold and Wait:** Process holds resources while waiting for more
3. **No Preemption:** Resources cannot be forcibly taken
4. **Circular Wait:** Chain of processes where each waits for next

Deadlock Handling

Table 2.6 Deadlock Handling Strategies

Strategy	Approach	Example
Prevention	Break one of 4 conditions	Request all resources at once (break hold-and-wait)
Avoidance	Safe state analysis	Banker's Algorithm
Detection & Recovery	Detect cycles, preempt	Wait-for graph analysis
Ignorance	Assume rare, reboot	Linux OOM killer

2.7 Memory Management

Virtual Memory

Virtual memory provides each process with the illusion of a large, private address space.

- **Virtual address:** What the program sees and uses
- **Physical address:** Actual location in RAM
- **MMU (Memory Management Unit):** Hardware that translates virtual to physical addresses
- **TLB (Translation Lookaside Buffer):** Cache for page table entries

Paging

- Memory divided into fixed-size pages (typically 4KB on x86_64)
- Virtual address split into: page number + offset
- Page table maps virtual page numbers to physical frame numbers
- Multi-level page tables reduce memory overhead
- x86_64 uses 4-level page tables (PGD → PUD → PMD → PTE)

Page Faults

1. CPU generates virtual address
2. TLB lookup fails (TLB miss)
3. Page table walk finds page not present
4. Page fault exception raised
5. OS handler runs:
6. If page on disk: find free frame (possibly evict), load from disk, update page table, restart instruction
7. If invalid address: SIGSEGV (segmentation fault)

Page Replacement Algorithms

Table 2.7 Page Replacement Algorithms

Algorithm	Strategy	Implementation	Performance
FIFO	First page in is first out	Queue	Simple, Belady's anomaly
Optimal	Replace page used farthest in future	Theoretical only	Best (unachievable)
LRU	Replace least recently used	Stack or counters	Good, near-optimal
Clock (Second Chance)	FIFO with use bit	Circular queue + reference bit	Approximates LRU efficiently
LFU	Replace least frequently used	Counter per page	Poor for changing locality

Linux Memory Management

- Buddy allocator for physical pages (prevents external fragmentation)
- Slab allocator for kernel objects (caches frequently allocated structures)
- Swapping: pages moved to disk when RAM full
- Swappiness parameter (0-100) controls swapping aggressiveness
- OOM (Out of Memory) Killer selects process to kill when memory exhausted

2.8 I/O Systems and Zero-Copy

I/O Techniques

Programmed I/O

CPU polls device status register until ready. Wastes CPU cycles.

Interrupt-Driven I/O

Device interrupts CPU when ready. CPU can do other work while waiting. Better but still requires CPU to copy data.

DMA (Direct Memory Access)

DMA OPERATION

1. CPU programs DMA controller with source, destination, and count
2. DMA controller transfers data directly between device and memory
3. DMA interrupts CPU when transfer complete
4. CPU only involved in setup and completion, not data movement

DMA is essential for high-speed I/O (network, disk) without consuming all CPU cycles.

Zero-Copy Techniques (CRITICAL FOR YOUR PROJECT)

Traditional Data Transfer (4 copies, 4 context switches)

1. Disk → Kernel pagecache (DMA, copy 1)
2. Kernel pagecache → User buffer (CPU copy, copy 2)
3. User buffer → Kernel socket buffer (CPU copy, copy 3)
4. Kernel socket buffer → NIC (DMA, copy 4)

Context switches: `read()` → kernel, return. `write()/send()` → kernel, return.

`sendfile()` (2 copies, 2 context switches)

1. Disk → Kernel pagecache (DMA, copy 1)
2. Kernel passes pagecache pages directly to socket (no user space copy!)
3. Kernel socket buffer → NIC (DMA, copy 2)

System call: `sendfile()` only - kernel handles everything internally.

SENDFILE(2) SYSTEM CALL

```
ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);
```

- Transfers data between file descriptors entirely in kernel space
- out_fd must be a socket (historically), in_fd must be a file supporting mmap
- Linux 2.4+: sendfile() uses DMA gather operations (true zero-copy to NIC)
- If NIC doesn't support scatter/gather, one kernel buffer copy still occurs
- Limitation: cannot modify data in transit (no encryption, no transformation)

CONNECTION TO UNMTP - SENDFILE() DEEP DIVE

Your UnMTP host uses sendfile() for Phase 1 (files $\geq$ 5GB). This is your strongest performance optimization. Be prepared to explain:

- **Why sendfile() is faster:** Eliminates 2 copies (kernel→user→kernel) and 2 context switches
- **Why 5GB threshold:** For large files, copy overhead dominates. For small files, tar streaming overhead is acceptable.
- **Why not mmap() + write():** mmap has TLB overhead and page fault costs. sendfile() is more efficient for file-to-socket transfer.
- **Kernel internals:** sendfile() uses splice() internally on Linux 2.6.17+. Pages are passed by reference using pipe buffers.
- **Limitation:** Cannot add encryption on-the-fly. Must encrypt file beforehand or use SSL_sendfile().

splice() - Generalized Zero-Copy

```
ssize_t splice(int fd_in, loff_t *off_in, int fd_out, loff_t *off_out,  
              size_t len, unsigned int flags);
```

- Transfers data between two file descriptors without copying to user space
- One of the fds must be a pipe
- sendfile() is implemented using splice() internally

mmap() + write() Alternative

```
addr = mmap(NULL, length, PROT_READ, MAP_PRIVATE, fd, 0);  
write(socket_fd, addr, length);
```

- Maps file into process virtual address space
- Pages loaded on demand (page faults)
- Still requires copy from pagecache to socket buffer (unlike sendfile)
- Useful when you need to inspect/modify data before sending

I/O Multiplexing

select()

- Monitors multiple file descriptors for readiness
- FD_SETSIZE limit (typically 1024)
- $O(n)$ scan of all fds on each call
- Modifies fd_set in place (must reinitialize)

poll()

- No fd limit
- Still $O(n)$ scan
- Uses array of pollfd structures

epoll() (Linux-specific)

- $O(1)$ for add/modify/remove operations
- $O(1)$ for waiting (returns only ready fds)
- Uses red-black tree internally
- Two modes: level-triggered (default) and edge-triggered
- Edge-triggered: only notifies on state change, must read/write until EAGAIN

STOP BOUNDARY - OPERATING SYSTEMS

Do NOT study: Detailed virtual memory page table walk algorithms, specific slab allocator internals, completely fair queuing (CFQ) I/O scheduler details, RT scheduling policy internals, kernel module programming, eBPF program writing. Know what eBPF does conceptually but not how to write programs.

Resources - Operating Systems

Books

- **Operating Systems: Three Easy Pieces (OSTEP)** by Remzi Arpaci-Dusseau - Free online, best OS textbook. Read all chapters.
- **Modern Operating Systems** by Andrew Tanenbaum - Reference text, good for concepts.
- **Linux Kernel Development** by Robert Love - For understanding Linux specifics.
- **The Linux Programming Interface** by Michael Kerrisk - Definitive reference for Linux system programming.

Courses

- Berkeley CS162: Operating Systems (YouTube)
- MIT 6.S081: Operating System Engineering (xv6 labs)
- CMU 15-410: Advanced Operating Systems

Expected Interview Questions - Operating Systems

Q1: Explain what happens when you call `sendfile()` in your project.

`sendfile()` is a kernel syscall that transfers data from a file descriptor to a socket descriptor entirely in kernel space. It avoids copying data through user space. Internally, Linux uses `splice()` which passes page cache pages by reference through pipe buffers. Only 2 copies occur (disk→pagecache via DMA, socket buffer→NIC via DMA) instead of 4 in the traditional path.

Q2: What is zero-copy and why does it matter for trading systems?

Zero-copy eliminates CPU-mediated data copies between kernel and user space. In trading, latency is directly tied to P&L. Every copy adds latency and consumes CPU cycles that could be used for order processing. Zero-copy techniques like `sendfile()`, `mmap()`, and kernel bypass (DPDK) are essential for high-frequency trading.

Q3: What is the difference between a process and a thread?

A process is an independent execution unit with its own address space, code, data, heap, and stack. A thread is a unit of execution within a process sharing code, data, and heap but with its own stack and registers. Context switch between threads is faster (same address space, no TLB flush). Processes are isolated; threads share memory and can easily communicate.

Q4: Explain virtual memory and paging.

Virtual memory gives each process the illusion of a large contiguous address space. Physical memory is divided into frames; virtual memory into pages (typically 4KB). The MMU translates virtual addresses to physical using page tables. The TLB caches recent translations. When a page is not in memory, a page fault occurs and the OS loads it from disk.

Q5: What is the difference between mutex and semaphore?

A mutex is a binary lock with ownership - only the thread that locked it can unlock it. Used for mutual exclusion. A semaphore is a counting variable that can be incremented/decremented by any thread. Binary semaphore is like mutex but without ownership. Semaphore can coordinate ordering (e.g., signal after producing); mutex protects critical sections.

Q6: What are the four conditions for deadlock? How do you prevent it?

Mutual exclusion, hold and wait, no preemption, circular wait. Prevention strategies: request all resources at once (break hold-and-wait), allow preemption (break no preemption), order resources and request in order (break circular wait). Avoidance: Banker's Algorithm. Detection: wait-for graph cycle detection.

Q7: Why does your file transfer use 8MB socket buffers? How did you choose this value?

8MB balances memory usage with throughput. For a LAN with RTT ~1ms and bandwidth 1Gbps, BDP = 125KB. 8MB is 64x the BDP, providing ample buffer for burst handling and smoothing out kernel scheduling jitter. For WAN, we'd calculate $BDP = \text{bandwidth} * \text{RTT}$ and set buffer accordingly. The 1MB write chunk on the client side matches typical disk write efficiency.

Q8: Explain the difference between select(), poll(), and epoll().

select(): O(n) scan, fd limit (1024), modifies fd_set. poll(): O(n) scan, no fd limit, separate events/revents. epoll(): O(1) operations, no fd limit, returns only ready fds, uses kernel event table (red-black tree). epoll is best for high-connection servers (10,000+ fds). For UnMTP with single connection, any would work.

Q9: What happens during a context switch?

Save current process state (registers, PC, stack pointer) to PCB. Update PCB state. Select next process from ready queue (scheduler). Restore new process state from PCB. Update memory management (possibly TLB flush). Jump to new PC. Cost: 1-10 microseconds plus cache/TLB pollution.

Q10: Explain the difference between paging and segmentation.

Paging divides memory into fixed-size pages (4KB). Internal fragmentation possible (last page). Simple memory management. Segmentation divides memory into variable-size segments (code, data, stack). Matches program structure. External fragmentation possible. Modern systems use paging; segmentation is largely historical (x86-64 uses flat segmentation).

Revision Checklist

- [] Can explain system call mechanism and overhead
- [] Can explain fork/exec/wait/exit with examples
- [] Can explain process states and transitions
- [] Can compare all scheduling algorithms
- [] Can explain mutex/semaphore/condition variable usage

- []Can identify and resolve deadlock scenarios
- []Can explain virtual memory translation (MMU, TLB, page tables)
- []Can explain page fault handling
- []Can explain sendfile() and zero-copy in detail
- []Can compare select/poll/epoll
- []Can explain DMA and its role in I/O

03

Python for Systems Programming

Difficulty: Medium **Importance:** Critical (P0) **Estimated Hours:** 38 **Week:** 5

WHY THIS MATTERS FOR D. E. SHAW

Python is the primary scripting language for automation, tooling, and glue code in trading infrastructure. D. E. Shaw uses Python extensively for internal tools, data processing, and system orchestration. Your interviewer will test whether you truly understand Python beyond syntax - they want to see deep knowledge of the standard library, memory management, and performance characteristics. Every module in your UnMTP project is fair game for deep questioning.

3.1 socket Module - Deep Dive

Core APIs to Master

Table 3.1 socket Module APIs

API	Purpose	Your Project Usage
<code>socket.socket(family, type)</code>	Create socket	AF_INET, SOCK_STREAM
<code>socket.bind(address)</code>	Bind to address:port	host: ('0.0.0.0', 5001)
<code>socket.listen(backlog)</code>	Listen for connections	<code>listen(1)</code>
<code>socket.accept()</code>	Accept connection	Returns (conn, addr)
<code>socket.connect(address)</code>	Connect to server	pull connects to host
<code>socket.sendfile(file, offset, count)</code>	Zero-copy send	Phase 1 transfers

<code>socket.makefile(mode, buffering)</code>	File wrapper	object	Phase 2 streaming	tar
<code>socket.setsockopt(level, optname, value)</code>	Set options	socket	SO_REUSEADDR, SNDBUF	
<code>socket.recv(bufsize)</code>	Receive data		Implicit makefile	via
<code>socket.close()</code>	Close socket		Cleanup in finally	

Socket Options Reference

Table 3.2 Important Socket Options

Option	Level	Purpose	UnMTP Usage
SO_REUSEADDR	SOL_SOCKET	Allow reuse of local address in TIME_WAIT	Yes - host restart
SO_SNDBUF	SOL_SOCKET	Send buffer size (kernel)	Yes - 8MB
SO_RCVBUF	SOL_SOCKET	Receive buffer size (kernel)	Yes - 8MB
SO_KEEPALIVE	SOL_SOCKET	Enable TCP keepalive probes	No - uses timeout
TCP_NODELAY	IPPROTO_TCP	Disable Nagle's algorithm	Not set
SO_LINGER	SOL_SOCKET	Control close() behavior	Not set
SO_RCVTIMEO	SOL_SOCKET	Receive timeout	Uses settimeout()

socketserver and selectors (Skip socketserver, Study selectors)

Do NOT study socketserver module - it is high-level and you already know the low-level APIs. DO study selectors:

- **selectors:** High-level I/O multiplexing, wraps select/poll/epoll/kqueue
- DefaultSelector chooses best implementation for platform
- Register sockets with events (EVENT_READ, EVENT_WRITE)
- select() returns list of (key, mask) tuples for ready sockets

3.2 struct Module - Binary Serialization

Format Characters to Know

Table 3.3 struct Format Characters

Format	C Type	Python Type	Size
x	pad byte	no value	1
c	char	bytes (length 1)	1
b	signed char	integer	1
B	unsigned char	integer	1
h	short	integer	2
H	unsigned short	integer	2
i	int	integer	4
I	unsigned int	integer	4
q	long long	integer	8
Q	unsigned long long	integer	8
f	float	float	4
d	double	float	8
s	char[]	bytes	variable

Endianness Prefixes

- **@:** Native order, native size, native alignment (default)
- **=:** Native order, standard size, no alignment
- **<:** Little-endian
- **>:** Big-endian (network byte order)
- **!:** Network byte order (same as >)

CONNECTION TO UNMTP - STRUCT USAGE

Your UnMTP uses `struct.pack('!HQ', path_len, filesize)` and `struct.unpack('!HQ', header)`. The `!` prefix means network byte order (big-endian). This ensures both Android (typically little-endian ARM) and Windows (little-endian x86) interpret the bytes consistently. Path length uses `H` (unsigned short, 2 bytes), file size uses `Q` (unsigned long long, 8 bytes). Total header: flag (1) + path_len (2) + filesize (8) = 11 bytes + path variable.

3.3 os and pathlib Modules

Essential os APIs

Table 3.4 os Module APIs for Systems Programming

API	Purpose	UnMTP Usage
<code>os.walk(top)</code>	Directory generator	tree Scan all files
<code>os.path.getsize(path)</code>	File size in bytes	Classify big files
<code>os.path.join(a, *p)</code>	Path concatenation	Build paths
<code>os.path.abspath(path)</code>	Absolute path	Normalize paths
<code>os.path.exists(path)</code>	Check existence	Validate input
<code>os.path.isdir(path)</code>	Is directory?	Validate input
<code>os.makedirs(path, exist_ok=True)</code>	Create directories	Create destination
<code>os.stat(path)</code>	File metadata	Size, permissions, times
<code>os.open(path, flags)</code>	Low-level file open	Used by <code>sendfile()</code>
<code>os.read(fd, n)</code>	Low-level read	Direct I/O
<code>os.close(fd)</code>	Close file descriptor	Cleanup

pathlib vs os.path

`pathlib` is the modern, object-oriented approach. Use `pathlib` for new code; know `os.path` for legacy and low-level work.

3.4 tarfile Module

Streaming Modes (Critical for UnMTP)

Table 3.5 tarfile Streaming Modes

Mode	Description	Use Case
'w '	Stream write (no seeking)	UnMTP host - writes to socket
'r '	Stream read (no seeking)	UnMTP pull - reads from socket
'w'	Write to file (seekable)	Standard archive creation
'r'	Read from file (seekable)	Standard archive extraction

CONNECTION TO UNMTP - STREAMING VS SEEKABLE

Your UnMTP uses streaming modes ('w|' and 'r|') because data flows through a TCP socket which is not seekable. You cannot rewind a socket stream. The '|' prefix means "no seeking allowed" - the tar file object operates purely as a stream. This is essential for your architecture because you write the tar directly to the socket without intermediate disk storage.

3.5 threading and multiprocessing

threading Module Essentials

- **Thread(target, args):** Create thread
- **start():** Begin execution
- **join(timeout):** Wait for completion
- **Lock/RLock:** Mutual exclusion
- **Semaphore:** Limited resource access
- **Event:** Simple signaling
- **Condition:** Complex synchronization
- **Queue:** Thread-safe FIFO

multiprocessing Module

- True parallelism (separate Python interpreters)
- Process class API similar to Thread
- Pipe and Queue for IPC
- Shared memory: Value, Array, Manager
- Pool for worker process pools

3.6 Advanced Python Concepts

Generators and Iterators

- Generators use `yield` to produce values lazily
- Memory efficient for large sequences
- Generator expressions: `(x for x in range(1000000))`
- Could be used in UnMTP for lazy file list generation

Context Managers

- `with` statement ensures cleanup (`__enter__` and `__exit__`)
- `contextlib` module: `@contextmanager` decorator
- UnMTP should use context managers for socket and file cleanup

Decorators

- Functions that wrap other functions
- `@functools.wraps` preserves metadata
- Useful for: logging, timing, retry logic, caching

Memory Management

- Reference counting: primary garbage collection mechanism
- Generational GC: handles reference cycles (objects referencing each other)
- `__del__` method called when reference count reaches zero
- `sys.getrefcount(obj)` to check reference count
- `gc` module for debugging circular references

3.7 Performance and Profiling

Profiling Tools

Table 3.6 Python Profiling Tools

Tool	Purpose	Usage
cProfile	Deterministic profiling	<code>python -m cProfile script.py</code>
line_profiler	Line-by-line profiling	<code>@profile</code> decorator
memory_profiler	Memory usage tracking	<code>@profile</code> for memory
timeit	Micro-benchmarking	<code>python -m timeit "code"</code>

Expected Interview Questions - Python

Q1: Why did you choose big-endian (!) for your binary protocol?

Network byte order is big-endian by convention (RFC 1700). Both sides know to expect big-endian regardless of their native architecture. Android ARM is typically little-endian, Windows x86 is little-endian. Using network byte order ensures interoperability without architecture detection.

Q2: What is the GIL and how does it affect your project?

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time. However, the GIL is released during I/O operations, so for UnMTP's network-bound workload, threading could still help if we added concurrent transfers. For true CPU parallelism, we'd use multiprocessing or asyncio.

Q3: Explain the difference between 'w|' and 'w' in tarfile.

'w|' is streaming mode - no seeking allowed, write sequentially to a stream. 'w' is file mode - requires a seekable file object. We use 'w|' because TCP sockets are not seekable. '|' prefix means the tar operates as a pure stream.

Q4: How does sendfile() work in Python's socket module?

`socket.sendfile(file, offset=0, count=None)` calls the OS `sendfile()` syscall which transfers data between file descriptors in kernel space. It avoids copying data through user space. Returns bytes sent. If OS doesn't support it, Python falls back to a read-send loop.

Q5: What is the difference between bytes and bytearray?

`bytes` is immutable - cannot be modified after creation. `bytearray` is mutable - supports item assignment. `bytearray` is more memory-efficient for building buffers incrementally. For your protocol headers, `bytes` is fine since headers are built once and not modified.

STOP BOUNDARY - PYTHON

Do NOT study: `asyncio` in depth (know basic concepts), `Cython`, Python C extension writing, Python bytecode internals, metaclasses (know what they are, not how to write them), descriptor protocol beyond `@property`. Skip all web frameworks (`Django`, `Flask`, `FastAPI`). Skip data science libraries (`numpy`, `pandas`).

04

Linux Systems

Difficulty: Medium-High **Importance:** High (P1) **Estimated Hours:** 32 **Weeks:** 3-5

WHY THIS MATTERS FOR D. E. SHAW

D. E. Shaw's trading infrastructure runs on Linux. As a Front Office Technical Associate, you will spend significant time on Linux servers debugging production issues, analyzing logs, monitoring systems, and writing shell scripts. Interviewers expect comfort with the command line and understanding of Linux internals. Your UnMTP host runs on Android/Termux (Linux), so your project experience with Linux paths, permissions, and socket behavior is directly relevant.

4.1 Filesystem and Navigation

Essential Commands (MUST KNOW)

Table 4.1 Linux Filesystem Commands

Command	Purpose	Key Options
ls	List files	-la, -lh, -t, -S, -R
cd	Change directory	~ (home), - (previous), .. (parent)
pwd	Print working directory	-P (physical, no symlinks)
find	Find files	-name, -type, -size, -mtime, -exec
locate	Find by database (faster)	-i (case insensitive)
df	Disk free space	-h (human readable)
du	Disk usage	-sh (summary, human)
mkdir	Create directory	-p (parents)

rm	Remove	-r (recursive), -f (force)
cp	Copy	-r (recursive), -p (preserve)
mv	Move/rename	-i (interactive)
ln	Link	-s (symbolic)
touch	Create/update timestamp	-t (specific time)

4.2 File Permissions and Ownership

Permission Model

```
-rwxr-xr-- 1 owner group 1234 Jul 1 10:00 file.txt
|  |  |
|  |  +--- Others: r--
|  +----- Group:  r-x
+----- Owner:  rwx

r = read (4), w = write (2), x = execute (1)
```

Special Permissions

- **SUID (4):** Execute as file owner (e.g., passwd)
- **SGID (2):** Execute as file group, new files inherit group
- **Sticky (1):** Only owner can delete (e.g., /tmp)

Commands

- `chmod 755 file` - numeric mode
- `chmod u+x file` - symbolic mode
- `chown user:group file` - change owner and group
- `umask` - default permission mask

4.3 Process Management

Essential Commands

Table 4.2 Process Management Commands

Command	Purpose	Key Options
ps	Process snapshot	aux, -ef, -o pid,ppid,cmd

top	Interactive process viewer	Sort by CPU, memory
htop	Enhanced top	F3 search, F9 kill
kill	Send signal	-9 (SIGKILL), -15 (SIGTERM), -HUP
killall	Kill by name	-9 name
pkill	Kill by pattern	-f (full command line)
nice	Run with modified priority	-n -10 (higher priority)
renice	Change priority of running process	-n 5 -p PID
pgrep	Find PID by name	-a (full command)
nohup	Run immune to hangups	nohup cmd &

Signals

Table 4.3 Important Linux Signals

Signal	Number	Default Action	Use
SIGHUP	1	Terminate	Hangup (terminal closed), reload config
SIGINT	2	Terminate	Ctrl+C, graceful interrupt
SIGQUIT	3	Core dump	Ctrl+\, debug quit
SIGKILL	9	Terminate (cannot catch)	Force kill, last resort
SIGTERM	15	Terminate	Graceful shutdown request
SIGSTOP	19	Stop (cannot catch)	Ctrl+Z, pause process
SIGCONT	18	Continue	Resume stopped process
SIGCHLD	17	Ignore	Child process state changed
SIGPIPE	13	Terminate	Write to pipe with no reader

4.4 Text Processing Tools

grep

- -i: case insensitive
- -v: invert match
- -r: recursive
- -n: line numbers
- -c: count matches
- -E: extended regex (egrep)
- -F: fixed strings (fgrep)
- -w: whole word match

sed (Stream Editor)

- sed 's/old/new/g' - substitute all occurrences
- sed -i '1d' - delete first line in-place
- sed -n '5,10p' - print lines 5-10
- sed '/pattern/d' - delete matching lines

awk

- awk '{print \$1}' - print first field
- awk -F: '{print \$NF}' - custom delimiter, last field
- awk '\$3 > 100 {print}' - conditional printing
- awk '{sum+=\$1} END {print sum}' - sum column
- awk 'BEGIN {OFS=","} {print \$1, \$2}' - output field separator

4.5 System Information and Monitoring

Commands

Table 4.4 System Monitoring Commands

Command	Purpose
free -h	Memory usage
vmstat 1	Virtual memory statistics
iostat -x 1	Disk I/O statistics

<code>mpstat -P ALL 1</code>	CPU usage per core
<code>sar -u 1</code>	System activity reporter
<code>netstat -tlnp / ss -tlnp</code>	Listening ports with PIDs
<code>lsof -i :5001</code>	Process using port 5001
<code>strace -p PID</code>	Trace system calls
<code>dmesg tail</code>	Kernel messages
<code>uptime</code>	Load average

4.6 Networking Commands

ip command (modern, replaces ifconfig/route)

- `ip addr show` - interface addresses
- `ip link show` - interface states
- `ip route show` - routing table
- `ip neigh show` - ARP table
- `ip addr add 192.168.1.10/24 dev eth0`

Other Network Commands

Table 4.5 Linux Network Commands

Command	Purpose
<code>ping -c 4 host</code>	ICMP echo test
<code>traceroute host</code>	Route path trace
<code>curl -v url</code>	HTTP client with verbose
<code>wget url</code>	File download
<code>scp file user@host:/path</code>	Secure copy
<code>rsync -avz src/ dest/</code>	Efficient sync
<code>ssh user@host</code>	Remote shell
<code>nc -zv host port</code>	Port connectivity test
<code>tcpdump -i eth0 port 5001</code>	Packet capture
<code>iptables -L -n -v</code>	Firewall rules

4.7 Systemd and Services

- `systemctl status service` - check service status
- `systemctl start/stop/restart service` - control service
- `systemctl enable/disable service` - auto-start on boot
- `systemctl list-units --type=service` - list services
- `journalctl -u service` - view service logs
- `journalctl -f` - follow logs in real-time
- `journalctl --since "1 hour ago"` - recent logs

4.8 Shell Scripting

Key Concepts

- Variables: `VAR="value"`, access with `$VAR`
- Conditionals: `if [ "$VAR" -eq 10 ]; then ... fi`
- Loops: `for i in {1..10}; do ... done`
- Functions: `myfunc() { ... }`
- Exit codes: 0=success, non-zero=failure
- Pipes: `cmd1 | cmd2` - stdout of `cmd1` to stdin of `cmd2`
- Redirection: `>` (overwrite), `>>` (append), `2>` (stderr), `>&` (both)
- Command substitution: `$(command)` or ``command``
- Test operators: `-e` (exists), `-f` (file), `-d` (directory), `-z` (empty string)

Expected Interview Questions - Linux

Q1: How do you find which process is using port 5001?

`ss -tlnp | grep 5001` or `lsof -i :5001`. Both show the PID and process name. `lsof` requires root to see all processes' sockets.

Q2: Explain the difference between SIGTERM and SIGKILL.

`SIGTERM` (15) is a graceful shutdown request - the process can catch it and clean up. `SIGKILL` (9) forcefully terminates immediately and cannot be caught or ignored. Always try `SIGTERM` first, use `SIGKILL` as last resort.

Q3: How do you trace system calls made by a process?

strace -p PID attaches to a running process. strace -e trace=network ./program filters for network syscalls. strace -c gives a summary count. This is invaluable for understanding what a program is actually doing at the kernel level.

Q4: What does SO_REUSEADDR do and why is it important?

SO_REUSEADDR allows binding to a port that is in TIME_WAIT state. Without it, restarting a server within 60 seconds of the last connection closing fails with "Address already in use". Essential for development and production services that restart.

STOP BOUNDARY - LINUX

Do NOT study: SELinux policy writing, Linux kernel module programming, eBPF program development, LVM internals, RAID configuration, advanced iptables/nftables rule writing, container internals (know basics of cgroups/namespaces). Know that containers exist and how they work conceptually, but not how to build them from scratch.

05

Debugging and Troubleshooting

Difficulty: Medium Importance: High (P1) Estimated Hours: 19 Week: 5

WHY THIS MATTERS FOR D. E. SHAW

Debugging production issues is arguably the core skill for a Front Office Technical Associate. Trading systems cannot afford downtime. When a market data feed disconnects or an order routing system slows down, you need systematic debugging skills to identify and resolve issues under pressure. D. E. Shaw interviewers often present hypothetical production scenarios and ask you to debug them. Your project debugging experience (connection refused, timeout handling, retry logic) demonstrates this mindset.

5.1 Debugging Methodology

General Principles

1. **Reproduce the issue:** If you can't reproduce it, you can't fix it reliably
2. **Isolate the problem:** Binary search through the system - network, OS, application
3. **Form a hypothesis:** Based on evidence, not guesses
4. **Test the hypothesis:** Make a prediction and verify
5. **Fix and verify:** Apply fix, confirm resolution
6. **Prevent recurrence:** Add monitoring, tests, or documentation

Layered Debugging for Network Applications

Table 5.1 Layered Debugging Checklist

Layer	Check	Tools
Application	Logs, error messages, stack traces	logging module, traceback
Library	API usage correctness	Documentation, source code
Socket	Socket options, buffer sizes	ss -tin, getsockopt

Transport	TCP state, retransmissions	netstat -s, ss -i
Network	Connectivity, routing	ping, traceroute, mtr
Data Link	ARP resolution, MAC addresses	ip neigh, arp -a
Physical	Cables, interface status	ip link, ethtool

5.2 Python Debugging Tools

pdb - Python Debugger

```
import pdb; pdb.set_trace() # Breakpoint
# Commands: n (next), s (step), c (continue), l (list), p (print), q (quit)
# Usage: python -m pdb script.py
```

Logging Best Practices

- Use the logging module, not print()
- Log levels: DEBUG (detail), INFO (normal), WARNING (unexpected), ERROR (failure), CRITICAL (system down)
- Include timestamps, process/thread ID, filename, line number
- Structured logging (JSON format) for production parsing
- Rotate logs to prevent disk fill (RotatingFileHandler)

Traceback Analysis

- Read from bottom up: the last line is the actual error
- The stack trace shows the call chain that led to the error
- Look for the innermost frame (your code, not library code)
- Common exceptions: ConnectionRefusedError, TimeoutError, PermissionError, OSError

5.3 Common Network Debugging Scenarios

Connection Refused

- Cause: No process listening on target port
- Check: ss -tlnp | grep PORT on server
- Check: Firewall blocking? iptables -L | grep PORT
- Check: Wrong IP or port?

Connection Timeout

- Cause: Network unreachable, firewall dropping, host down

- Check: ping host
- Check: traceroute host
- Check: Firewall rules on both sides

Broken Pipe

- Cause: Remote end closed connection while local still writing
- Common in: Unhandled client disconnect
- Fix: Try/except around send operations, handle EPIPE

Slow Transfer

- Check: Bandwidth: iperf3 between hosts
- Check: RTT: ping
- Check: TCP window size: ss -tin
- Check: Retransmissions: netstat -s | grep retrans
- Check: CPU: top
- Check: Disk I/O: iostat

5.4 Performance Debugging

CPU Profiling

- cProfile: `python -m cProfile -s cumulative script.py`
- line_profiler: @profile decorator, `kernprof -l -v script.py`

Memory Profiling

- memory_profiler: @profile, `python -m memory_profiler script.py`
- tracemalloc: Standard library memory tracking
- objgraph: Object reference visualization

Network Performance

- iperf3: Bandwidth testing between hosts
- netperf: Network performance benchmarking
- tc (traffic control): Simulate network conditions

Expected Interview Questions - Debugging

Q1: Your file transfer suddenly slows from 100 MB/s to 1 MB/s. How do you debug?

Check CPU (top), memory pressure (free), disk I/O (iostat), network retransmissions (netstat -s), TCP window size (ss -tin), cable/interface errors (ethtool -S). Use tcpdump to see if packets are flowing. Check if the 5GB threshold logic is misclassifying files. Profile with cProfile to find hot spots.

Q2: A client gets "Connection Refused" every time. Walk through your debugging.

1) Verify server is running (ps). 2) Check port is listening (ss -tlnp). 3) Verify client IP and port are correct. 4) Check firewall rules (iptables -L). 5) Try connecting locally on server (nc -vz localhost 5001). 6) Check if SO_REUSEADDR is set. 7) Verify network connectivity (ping). 8) Capture packets (tcpdump) to see if SYN arrives.

Q3: How do you debug a memory leak in a long-running Python process?

Use tracemalloc to track allocations over time. Take snapshots and compare: snapshot2.compare_to(snapshot1, 'lineno'). Use objgraph to find growing object types. Check for unclosed files, unjoined threads, or growing caches. Use memory_profiler for line-by-line analysis.

06

Data Structures & Algorithms

Difficulty: Medium **Importance:** High (P1) **Estimated Hours:** 32 **Week:** 6

WHY THIS MATTERS FOR D. E. SHAW

D. E. Shaw does ask DSA questions, but they are practical rather than competitive programming-style. They want to see structured problem-solving, not clever tricks. Focus on medium difficulty. Hash map optimizations, two-pointer techniques, and binary search are particularly relevant because trading systems constantly search, match, and aggregate data.

Study Approach

Target: **150-200 problems**, Easy to Medium only. No hard problems unless you have extra time. Focus on quality over quantity - understand the pattern, not just the solution.

6.1 Arrays and Strings

Study Until:

- Prefix Sum and Difference Arrays
- Kadane's Algorithm (maximum subarray)
- Sliding Window (fixed and variable size)
- Two Pointer technique
- Binary Search on arrays (including rotated)
- Merge intervals

STOP BOUNDARY - ARRAYS

Stop after Medium difficulty. Do NOT study: segment trees, Fenwick trees, sparse tables, advanced range query structures.

Practice: 40 problems. **Difficulty:** Easy + Medium only. **Interview Focus:** Hashing-based optimizations, sliding window for streaming data.

6.2 Hashing

Study Until:

- Hash map/set usage patterns
- Two-sum and variations
- Frequency counting
- Caching and memoization
- Collision resolution (chaining, open addressing) - conceptually

Practice: 25 problems. **Key Skill:** $O(n)$ solutions using hash maps instead of $O(n^2)$ brute force.

6.3 Linked Lists

Study Until:

- Reverse a linked list (iterative and recursive)
- Detect cycle (Floyd's algorithm)
- Find merge point
- Remove Nth node from end
- Merge two sorted lists

Practice: 15 problems. **Key Skill:** Pointer manipulation, edge cases (empty list, single node).

6.4 Stacks and Queues

Study Until:

- Implement stack with queues, queue with stacks
- Valid parentheses
- Monotonic stack (next greater element)
- LRU Cache (hash map + doubly linked list)
- Min stack ($O(1)$ getMin)

Practice: 20 problems. **Key Skill:** Understanding when LIFO vs FIFO is appropriate.

6.5 Trees and Binary Search Trees

Study Until:

- DFS traversals (preorder, inorder, postorder)
- BFS/level-order traversal
- Validate BST
- LCA (Lowest Common Ancestor)
- Tree height and diameter
- Basic AVL/Red-Black tree concepts (self-balancing, rotations)

STOP BOUNDARY - TREES

Do NOT study: advanced tree DP, heavy-light decomposition, centroid decomposition, treaps, splay trees. Know that self-balancing trees exist and why ($O(\log n)$ guaranteed), but not implementation details.

Practice: 25 problems. **Interview Focus:** BST property applications, recursive thinking.

6.6 Heaps/Priority Queues

Study Until:

- Min/max heap properties
- Heapify operation
- Top K elements pattern
- Merge K sorted lists
- Heap sort (conceptually)

Practice: 15 problems. **Key Skill:** Recognizing "top K" and "streaming median" problems.

6.7 Graphs

Study Until:

- BFS and DFS traversal
- Detect cycle (directed and undirected)
- Topological sort (Kahn's algorithm)
- Dijkstra's shortest path (basic implementation)
- Union-Find (disjoint set)

STOP BOUNDARY - GRAPHS

Do NOT study: Bellman-Ford, Floyd-Warshall, A* search, network flow (max flow, min cut), strongly connected components (Tarjan), articulation points, minimum spanning tree algorithms in detail.

Practice: 20 problems. **Interview Focus:** BFS/DFS pattern recognition, connected components.

6.8 Binary Search

Study Until:

- Standard binary search and variations
- Search in rotated sorted array
- Find first/last occurrence
- Binary search on answer (minimize maximum)

Practice: 15 problems. **Key Skill:** Template binary search that handles all boundary cases.

6.9 Sorting and Searching

Know These Algorithms:

Table 6.1 Sorting Algorithms

Algorithm	Time (Avg)	Time (Worst)	Space	Stable
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

6.10 Recursion and Backtracking

Study Until:

- Subsets and permutations
- N-Queens (conceptually)
- Basic backtracking template

STOP BOUNDARY - RECURSION

Do NOT study: advanced backtracking optimizations, constraint satisfaction problems, dancing links algorithm.

Practice: 10 problems. **Key Skill:** Recognizing when recursion is appropriate vs iterative.

Resources - DSA

- **LeetCode:** Primary practice platform. Focus on "Top Interview 150" list.
- **GeeksforGeeks:** Good for concept explanations (not for problem practice).
- **Introduction to Algorithms (CLRS):** Reference for algorithms you don't understand.
- **NeetCode Roadmap:** Structured topic progression.

07

Object-Oriented Programming

Difficulty: Low-Medium Importance: Medium (P2) Estimated Hours: 6 Week: 7

7.1 Core Concepts

Four Pillars of OOP

Table 7.1 OOP Pillars

Pillar	Definition	Python Example
Encapsulation	Bundle data and methods, hide internal state	Private attributes (<code>_name</code>), getters/setters (<code>@property</code>)
Inheritance	Derive new classes from existing ones	<code>class Child(Parent):</code>
Polymorphism	Same interface, different implementations	Duck typing, method overriding
Abstraction	Hide complexity, show essential features	ABC module, abstract methods

7.2 SOLID Principles

Table 7.2 SOLID Principles

Principle	Definition	Violation Example
S - Single Responsibility	Class should have one reason to change	God class that does everything

O - Open/Closed	Open for extension, closed for modification	Modifying existing code for new feature
L - Liskov Substitution	Derived class substitutable for base	Subclass that throws on base method
I - Interface Segregation	Small specific interfaces over large general	Interface with methods clients don't need
D - Dependency Inversion	Depend on abstractions, not concretions	Direct instantiation of dependencies

7.3 Python OOP Specifics

- `__init__`: Constructor
- `__str__` / `__repr__`: String representations
- `__eq__` / `__hash__`: Equality and hashability
- `__enter__` / `__exit__`: Context managers
- `@property`: Getter without method call syntax
- `@staticmethod` / `@classmethod`: Non-instance methods
- `@dataclass`: Auto-generate `__init__`, `__repr__`, `__eq__`
- `abc.ABC`: Abstract base classes
- `super()`: Call parent class methods
- **MRO (Method Resolution Order)**: C3 linearization for multiple inheritance

CONNECTION TO UNMTP - OOP REFACTORING

Your UnMTP is procedural (functions). If asked about OOP, explain that for a 400-line tool, procedural is appropriate. But for extension, you could refactor into classes: `TransferServer` (host), `TransferClient` (pull), `ProtocolHandler` (serialization), and `FileClassifier` (threshold logic). This demonstrates OOP thinking without over-engineering.

STOP BOUNDARY - OOP

Do NOT study: design patterns in exhaustive detail (know Singleton, Factory, Observer, Strategy, Adapter). Do NOT study UML. Know that patterns exist and when to use them, but focus on SOLID principles and clean code.

08

SQL

Difficulty: Low-Medium **Importance:** Medium (P2) **Estimated Hours:** 6 **Week:** 7

8.1 Core Concepts

Must Know

- SELECT, INSERT, UPDATE, DELETE
- WHERE, ORDER BY, LIMIT, OFFSET
- JOINS: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF
- GROUP BY, HAVING, aggregate functions (COUNT, SUM, AVG, MIN, MAX)
- Subqueries: correlated and non-correlated
- UNION, INTERSECT, EXCEPT
- Indexes: B-tree, clustered vs non-clustered, composite
- Transactions: ACID properties
- Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable
- Normalization: 1NF, 2NF, 3NF, BCNF

ACID Properties

Table 8.1 ACID Properties

Property	Description	Real-World Issue
Atomicity	All operations complete or none	Partial transfer, corrupted state
Consistency	Database remains valid	Constraint violations
Isolation	Concurrent transactions don't interfere	Lost updates, dirty reads

Durability

Committed data survives failures

Data loss on crash

Transaction Isolation Levels and Problems

Table 8.2 Isolation Levels

Level	Dirty Read	Non-Repeatable	Phantom
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Practice

20 problems on LeetCode or HackerRank. Focus on JOINS, GROUP BY with HAVING, and subqueries. Practice writing queries without looking up syntax.

STOP BOUNDARY - SQL

Do NOT study: query optimization internals (cost-based optimization, execution plans), stored procedures, triggers, cursors, window functions (know what they are), CTEs (know what they are), database replication internals, sharding strategies.

09

Git

Difficulty: Low **Importance:** Low-Medium (P3) **Estimated Hours:** 3 **Week:** 7

9.1 Commands to Know

Essential

- `git init`, `clone`, `status`, `add`, `commit`, `push`, `pull`
- `git branch`, `checkout`, `switch`, `merge`
- `git stash` (`push`, `pop`, `list`, `drop`)
- `git log` (`--oneline`, `--graph`, `--decorate`)
- `git diff`, `git show`

Intermediate (Interview Level)

- `git rebase` (`interactive: -i`) - replay commits on top of another branch
- `git cherry-pick` - apply specific commits
- `git reset` (`--soft`, `--mixed`, `--hard`) - undo changes at different levels
- `git revert` - create commit that undoes previous commit
- `git reflog` - recover lost commits
- `git bisect` - binary search for bug introduction

9.2 Key Concepts

- **Three areas:** Working directory → Staging (index) → Repository (HEAD)
- **Merge vs Rebase:** Merge preserves history (non-linear), rebase creates linear history
- **Fast-forward:** Direct path merge, no new commit needed
- **Merge conflict:** Both branches modified same lines - manual resolution
- **.gitignore:** Patterns for untracked files

9.3 Git Internals (Know Conceptually)

- Git is a content-addressable filesystem (key-value store)
- Objects: blob (file content), tree (directory), commit (snapshot + metadata), tag
- SHA-1 hash of content = object name
- Refs: branches (mutable), tags (immutable)
- HEAD: pointer to current branch/commit

STOP BOUNDARY - GIT

Do NOT study: git hooks implementation, git submodule/subtree workflows, git LFS, advanced git plumbing commands, patch-based workflows.

10

Computer Architecture

Difficulty: Medium **Importance:** Low-Medium (P3) **Estimated Hours:** 5 **Week:** 7

10.1 Concepts to Master

CPU Architecture Basics

- **Von Neumann Architecture:** Stored program concept, shared memory for data and instructions
- **Registers:** PC (program counter), SP (stack pointer), general purpose, status flags
- **ALU:** Arithmetic Logic Unit - performs arithmetic and bitwise operations
- **Control Unit:** Directs operation, fetches and decodes instructions
- **Cache hierarchy:** L1 (fastest, smallest), L2, L3 (slowest, largest, shared)
- **Cache line:** Unit of cache transfer (typically 64 bytes)
- **Temporal locality:** Recently accessed items likely to be accessed again
- **Spatial locality:** Nearby items likely to be accessed together

Memory Hierarchy

Speed (fast to slow)	Size (small to large)	Cost (high to low)
Registers -----	~KB -----	Highest
L1 Cache -----	~32-64KB per core	
L2 Cache -----	~256-512KB per core	
L3 Cache -----	~8-64MB shared	
RAM -----	~8-512GB	
SSD/HDD -----	~256GB - TBs	
Network Storage -----	PBs -----	Lowest

Cache Coherency (Basic Awareness)

- MSI, MESI, MOESI protocols for multi-core cache consistency
- False sharing: independent variables on same cache line cause unnecessary invalidation

Endianness

- **Big-endian:** Most significant byte at lowest address (network byte order)
- **Little-endian:** Least significant byte at lowest address (x86, ARM)
- Your UnMTP uses big-endian (! in struct) for wire protocol compatibility

64-bit vs 32-bit

- 64-bit: larger address space (theoretical 16 exabytes), more registers, larger data paths
- Pointer size: 8 bytes on 64-bit vs 4 bytes on 32-bit (affects memory usage)
- `Size_t` and pointer types match architecture word size

STOP BOUNDARY - COMPUTER ARCHITECTURE

Do NOT study: pipelining details, branch prediction algorithms, out-of-order execution details, speculative execution (Spectre/Meltdown mechanism), microarchitecture of specific CPUs, assembly programming, FPGA design.

11

System Design Basics

Difficulty: Medium **Importance:** Medium (P2) **Estimated Hours:** 13 **Week:** 7

WHY THIS MATTERS FOR D. E. SHAW

System design questions at D. E. Shaw for this role are practical, not theoretical. They will ask you to extend your project or design similar systems. "Design a file transfer system" becomes easy because you've built one. Be ready to discuss tradeoffs, scaling, failure handling, and monitoring.

11.1 Key Concepts

Scalability

- **Horizontal scaling:** Add more machines (scale out)
- **Vertical scaling:** Bigger machine (scale up) - has limits
- **Stateless vs stateful:** Stateless services easier to scale

Load Balancing

- Round-robin, least connections, IP hash, weighted
- Layer 4 (transport) vs Layer 7 (application) load balancing
- Health checks to detect failed backends

Caching

- Cache aside (lazy loading), read-through, write-through, write-behind
- Cache eviction: LRU, LFU, TTL
- Cache invalidation: "There are only two hard things in CS..."

Database Scaling

- Read replicas for read scaling
- Sharding for write scaling

- CAP theorem: Consistency, Availability, Partition tolerance - pick two

11.2 System Design Questions to Prepare

Design a file transfer system like your UnMTP but production-ready.

Key components: metadata service (file registry), transfer service (orchestrates transfers), storage backend, notification service. Add TLS encryption, authentication (OAuth/JWT), chunked transfer with resume, checksum verification (SHA-256), parallel streams, progress tracking, retry with exponential backoff, monitoring (Prometheus/Grafana), and horizontal scaling of transfer nodes.

How would you scale your file transfer to handle 1000 concurrent transfers?

1) Connection pool instead of single connection. 2) Async I/O (asyncio or epoll-based). 3) Worker pool for CPU-intensive tasks (checksums). 4) Load balancer across multiple transfer servers. 5) Shared storage backend (S3/GCS) or peer-to-peer. 6) Rate limiting to prevent overload. 7) Queue-based architecture with message broker (RabbitMQ/SQS).

Design a URL shortener.

API: create(shorten) and redirect. Database: mapping of short key to URL. Short key generation: base62 encoding of auto-increment ID or hash. Cache: Redis for hot URLs. Scaling: read replicas, sharding by key prefix. Analytics: separate write path for click tracking.

STOP BOUNDARY - SYSTEM DESIGN

Do NOT study: microservices patterns in detail, Kubernetes architecture, service mesh (Istio), CQRS/Event sourcing, complex distributed consensus (Raft/Paxos implementation). Know what these are conceptually but not implementation.

12

Concurrency

Difficulty: Medium-High **Importance:** Medium (P2) **Estimated Hours:** 10 **Week:** 5

12.1 Concurrency Models

Table 12.1 Concurrency Approaches in Python

Approach	Mechanism	Best For	Limitation
threading	OS threads, GIL	I/O-bound tasks	No true parallelism (CPU-bound)
multiprocessing	Separate processes	CPU-bound tasks	High memory overhead
asyncio	Event loop, Many coroutines	Many connections	I/O Requires async ecosystem

12.2 Key Concepts

- **Race condition:** Outcome depends on execution order of threads/processes
- **Deadlock:** Circular waiting (see OS chapter)
- **Starvation:** Thread never gets resource
- **Livelock:** Threads keep responding to each other without progress
- **Atomic operation:** Indivisible operation (CPU instruction level)
- **Lock ordering:** Always acquire locks in same order to prevent deadlock
- **Thread pool:** Pre-created threads to avoid creation overhead
- **Futures/Promises:** Represent result of async computation

12.3 asyncio Basics

- `async def` - defines a coroutine

- `await` - yield control, wait for result
- `asyncio.run()` - entry point
- `asyncio.create_task()` - schedule coroutine
- `asyncio.gather()` - run multiple coroutines concurrently
- `asyncio.Lock()`, `asyncio.Semaphore()` - async synchronization

CONNECTION TO UNMTP - CONCURRENCY EXTENSION

Your UnMTP uses a single connection. To add concurrency, options: (1) Threading: multiple simultaneous transfers (I/O bound, GIL released during socket I/O). (2) `asyncio`: single-threaded handling of many clients. (3) multiprocessing: parallel transfers across CPU cores. For a production system, `asyncio` with `epoll` provides the best scalability for many concurrent connections.

13

File Systems

Difficulty: Medium **Importance:** Medium (P2) **Estimated Hours:** 6 **Week:** 6

13.1 Key Concepts

File System Components

- **Superblock:** File system metadata (size, block size, free blocks)
- **Inode:** Metadata for each file (size, permissions, timestamps, pointers to data blocks)
- **Data blocks:** Actual file content storage
- **Directory:** Special file containing filename-to-inode mappings

ext4 (Linux Default)

- Extents: contiguous block allocation (better for large files than indirect blocks)
- Journaling: logs metadata changes for crash recovery (ordered mode default)
- Delayed allocation: improves performance and reduces fragmentation
- 64-bit support, up to 1 exabyte file systems

File Types in Linux

- - : Regular file
- d : Directory
- l : Symbolic link
- c : Character device
- b : Block device
- s : Socket
- p : Named pipe (FIFO)

Important File System Concepts

- **Hard link:** Multiple directory entries point to same inode. Same file system only.
- **Symbolic link:** Special file containing path to another file. Cross filesystem.
- **Mount:** Attach filesystem to directory tree

- **df -i**: Check inode usage (can run out of inodes before disk space)
- **atime**: Access time (often disabled for performance: noatime mount option)

13.2 TAR Format (Relevant to UnMTP)

- TAR = Tape Archive. Originally for sequential tape storage.
- Streaming tar (mode='w|') writes headers and file content sequentially without seeking.
- Each file: 512-byte header + padded file content (multiple of 512 bytes)
- Header contains: filename, mode, uid, gid, size, mtime, checksum, type flag
- End of archive: two 512-byte blocks of zeros

CONNECTION TO UNMTP - TAR STREAMING

Your UnMTP streams tar over TCP because tar was designed for sequential I/O (tapes). This is a perfect match for network sockets. The filter callback in `tar.add()` allows excluding already-transferred big files. Understanding that tar pads to 512-byte boundaries helps explain overhead for small files.

14

Networking Tools

Difficulty: Medium **Importance:** Medium (P2) **Estimated Hours:** 5 **Week:** 1-2

This chapter complements Chapter 1 (Computer Networks) with hands-on tooling. Every tool listed here should be practiced in a lab environment.

14.1 Packet Capture and Analysis

tcpdump

```
tcpdump -i eth0           # capture on interface
tcpdump -i any port 5001   # filter by port
tcpdump -i any host 192.168.1.1 # filter by host
tcpdump -nn -A port 80     # no DNS, ASCII payload
tcpdump -w capture.pcap    # write to file
tcpdump -r capture.pcap    # read from file
tcpdump -c 100 -s 0 -w out.pcap # 100 packets, full snaplen
```

Wireshark

- Display filters: `tcp.port == 5001`, `ip.addr == 192.168.1.1`, `tcp.flags.syn == 1`
- Follow TCP stream: Right-click → Follow → TCP Stream
- Expert info: Analyze → Expert Info (warnings, errors, notes)
- IO graphs: Statistics → IO Graphs (throughput over time)
- Flow graph: Statistics → Flow Graph (sequence diagram)

14.2 Connectivity Testing

iperf3

```
# Server
iperf3 -s -p 5001

# Client
iperf3 -c server_ip -p 5001 -t 30 # 30 second test
iperf3 -c server_ip -u -b 1G      # UDP test at 1 Gbps
iperf3 -c server_ip -P 10         # 10 parallel streams
```

netcat (nc)

```
# Listen on port
nc -l 5001

# Connect to port
nc host 5001

# Transfer file (basic)
# Receiver: nc -l 5001 > received_file
# Sender:   nc receiver_ip 5001 < file_to_send

# Port scan
nc -zv host 5000-5010
```

14.3 Network Monitoring

ss (socket statistics)

```
ss -tlnp      # TCP listening sockets with processes
ss -tin       # TCP internal info (buffer sizes, timers)
ss -s         # socket summary
ss -tn state established # only established connections
```

iftop / nethogs

- iftop: bandwidth usage by connection (like top for network)
- nethogs: bandwidth usage by process

14.4 Firewall Basics

iptables (legacy but still common)

```
iptables -L -n -v          # list rules
iptables -A INPUT -p tcp --dport 5001 -j ACCEPT # allow port
iptables -A INPUT -p tcp --dport 5001 -s 192.168.1.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 5001 -j DROP   # drop others
```

firewalld (RHEL/CentOS)

```
firewall-cmd --list-all
firewall-cmd --add-port=5001/tcp --permanent
firewall-cmd --reload
```

ufw (Ubuntu)

```
ufw allow 5001/tcp
ufw status
```

STOP BOUNDARY - NETWORKING TOOLS

Do NOT study: advanced Wireshark dissector writing, custom tcpdump filter expressions beyond basic, network simulation tools (GNS3), load balancer configuration (HAProxy/nginx beyond basics), advanced nftables/iptables chains and tables.

15

Security Basics

Difficulty: Medium **Importance:** Medium (P2) **Estimated Hours:** 4 **Week:** 7

15.1 Core Concepts

Cryptography Primitives

- **Symmetric encryption:** Same key for encryption and decryption. AES (128, 256), ChaCha20. Fast, used for bulk data.
- **Asymmetric encryption:** Public/private key pair. RSA, ECC. Slow, used for key exchange and signatures.
- **Hash functions:** One-way, fixed output. SHA-256, SHA-3. Used for integrity verification.
- **HMAC:** Hash-based message authentication code. Provides integrity + authenticity.
- **Digital signatures:** Sign with private key, verify with public key. Proves authenticity.

TLS/SSL

- TLS 1.2: 2-RTT handshake, RSA or DHE key exchange, AES-GCM or ChaCha20-Poly1305
- TLS 1.3: 1-RTT handshake (0-RTT with resumption), removes RSA key exchange (forward secrecy only)
- Certificate chain: leaf → intermediate CA → root CA
- Certificate validation: not expired, correct domain, trusted issuer, not revoked (CRL/OCSP)

Authentication and Authorization

- **Authentication:** Verify identity (who you are) - passwords, tokens, certificates, biometrics
- **Authorization:** Verify permissions (what you can do) - RBAC, ACLs, policies
- **OAuth 2.0:** Delegated authorization framework. Authorization code flow, client credentials flow.
- **JWT:** JSON Web Token. Contains claims, signed (JWS) or encrypted (JWE). Stateless auth.

Common Vulnerabilities

Table 15.1 Common Security Vulnerabilities

Vulnerability	Description	Mitigation
Buffer Overflow	Write past buffer boundary	Bounds checking, safe languages
Injection (SQL, Command)	Untrusted input as code	Parameterized queries, input validation
Path Traversal	Access files outside intended directory	Canonicalize paths, chroot
Man-in-the-Middle	Intercept communication	TLS, certificate pinning
Replay Attack	Resend captured valid request	Nonces, timestamps
DoS/DDoS	Overload service	Rate limiting, CDN, WAF

15.2 Security and Your UnMTP Project

UNMTP SECURITY LIMITATIONS AND IMPROVEMENTS

Current limitations:

- No encryption: All data sent in plaintext
- No authentication: Any client can connect
- No integrity verification: No checksums or hashes
- No rate limiting: Client can request unlimited data
- Path traversal risk: `tar.extractall()` without path validation

Improvements to discuss:

- Add TLS 1.3 with mutual authentication (client certificates)
- Implement HMAC-SHA256 for packet integrity verification
- Add authentication token in protocol header
- Validate extracted paths to prevent directory traversal
- Add rate limiting and connection quotas
- Implement transfer logging and audit trail

Expected Interview Questions - Security

Q1: Your protocol has no encryption. How would you add it?

Options: (1) TLS wrapper around TCP connection (ssl.wrap_socket in Python). (2) Encrypt files before transfer with AES-256-GCM, send key through separate channel. (3) Use SSH tunnel or VPN. TLS is easiest - minimal code changes, proven security. Would need to benchmark impact on throughput.

Q2: How would you prevent directory traversal in your file transfer?

Validate all paths before extraction: `os.path.commonpath([dest_dir, target_path]) == dest_dir`. Reject paths containing '..'. Use `os.path.abspath()` followed by prefix check. In Python 3.12+, `tarfile` has `filter='fully_trusted'/'tar'/'data'` options for security.

Q3: What is the difference between authentication and authorization?

Authentication verifies identity (who you are) - passwords, certificates, tokens. Authorization determines permissions (what you can do) - access control lists, roles. Authentication always comes first. A valid user (authenticated) may still lack permission (not authorized) for a specific resource.

STOP BOUNDARY - SECURITY

Do NOT study: advanced cryptography (elliptic curve math, lattice-based crypto), penetration testing techniques, detailed TLS 1.3 handshake state machine, certificate authority infrastructure internals, threat modeling frameworks (STRIDE, DREAD). Know what these are but not implementation details.

16

Behavioral Interview Preparation

Difficulty: Low **Importance:** High **Estimated Hours:** 4 **Week:** 8

WHY THIS MATTERS

D. E. Shaw is a collaborative, intellectually intense environment. Behavioral interviews assess culture fit, communication skills, and how you handle pressure. They want people who can work alongside traders and researchers under tight deadlines.

16.1 STAR Method

For every behavioral question, structure your answer:

- **Situation:** Brief context (2-3 sentences)
- **Task:** What you needed to accomplish
- **Action:** What YOU specifically did (focus on your contributions)
- **Result:** Quantifiable outcome, lessons learned

16.2 Questions to Prepare

Leadership and Initiative

1. Tell me about a time you took ownership of a project.
2. Describe a situation where you identified a problem before anyone else.
3. How have you improved a process or system in a previous role?

Collaboration

1. Tell me about a time you had a conflict with a teammate. How did you resolve it?
2. Describe a project where you had to work with people from different backgrounds.
3. How do you handle disagreements about technical approaches?

Problem Solving

1. Tell me about the most challenging technical problem you've solved.
2. Describe a time you had to debug something under time pressure.
3. How do you approach problems you've never seen before?

Adaptability

1. Tell me about a time priorities changed suddenly. How did you adapt?
2. Describe a project that didn't go as planned. What did you learn?
3. How do you handle tight deadlines?

Why D. E. Shaw

1. Why do you want to work at D. E. Shaw specifically?
2. What interests you about the Front Office Technical Associate role?
3. Where do you see yourself in 3-5 years?
4. What do you know about our company culture?

16.3 Sample Answer - Why D. E. Shaw

SAMPLE ANSWER FRAMEWORK

"I'm drawn to D. E. Shaw because of the unique intersection of technology and finance. My background in systems programming and networking directly applies to the infrastructure challenges of a quantitative trading firm. I'm excited about working on systems where performance and reliability directly impact outcomes. The firm's reputation for technical excellence and the opportunity to learn from world-class engineers and researchers is exactly what I'm looking for at this stage of my career."

16.4 Sample Answer - Most Challenging Problem

SAMPLE ANSWER (YOUR UNMTP PROJECT)

"The most challenging technical problem I solved was optimizing large file transfers between Android and Windows. The initial approach copied data through user space, which was slow for multi-gigabyte files. I researched kernel-level zero-copy mechanisms and implemented `sendfile()`, which eliminated unnecessary memory copies. I also had to design a custom binary protocol to handle both large files (zero-copy) and small files (tar streaming) efficiently. The result was a 3-5x speedup for large files, and the system now transfers files at near-disk-speed over LAN."

17

Resume Discussion

Difficulty: Low **Importance:** High **Estimated Hours:** 2 **Week:** 8

17.1 Resume Preparation

How to Describe Your UnMTP Project

Lead with impact and technical depth. Do NOT say "I made a file transfer app."

STRONG DESCRIPTION

"Designed and implemented a hybrid file transfer protocol in Python that optimizes large-file transfers using kernel-level zero-copy `sendfile()` while streaming smaller files as a TAR archive over a custom TCP wire protocol. The system features automatic network discovery, cross-platform support between Android (Termux) and Windows, and achieves near-disk-speed throughput over LAN. Built entirely with Python standard library modules (`socket`, `struct`, `tarfile`) with zero external dependencies."

Keywords to Include

- TCP/IP socket programming
- Binary wire protocol design
- Zero-copy I/O (`sendfile`)
- Cross-platform development
- Performance optimization
- Client-server architecture
- Network programming
- Systems programming

17.2 Expected Resume Questions

Walk me through your strongest project.

Prepare a 2-minute and 5-minute version. Start with the problem, your approach, technical decisions, challenges overcome, and results. Be ready to go deep on any aspect they ask about.

Why Python for this project? Would you choose it again?

"Python's standard library has excellent socket and system programming support (socket, struct, tarfile). Rapid prototyping allowed me to validate the approach quickly. For production at scale, I might add a Rust or C++ component for the hot path while keeping Python for orchestration, or use PyPy for better performance."

What would you do differently if you started over?

"I'd add TLS from the start for security, use asyncio for concurrent transfers, add proper logging with structured output, implement checksum verification, and design the protocol with extensibility in mind (version field in header). I'd also add unit tests and integration tests from the beginning."

18

Project Discussion: 100+ Interview Questions

Difficulty: Variable **Importance:** CRITICAL **Estimated Hours:** 26 **Week:** 8

THIS CHAPTER IS YOUR SECRET WEAPON

This is the most important section of the entire handbook. Your UnMTP project is your strongest asset. D. E. Shaw interviewers will spend significant time discussing your project in depth. They will probe every design decision, challenge your assumptions, and ask how you would extend it. Every question below has been crafted based on real interview patterns at quantitative trading firms and tech companies. Write out answers to all of them. Practice speaking them aloud.

Architecture Questions (1-10)

Q1: Give me a high-level overview of your file transfer system.

UnMTP is a two-phase client-server file transfer system. The host (server) runs on the source device, typically Android via Termux. It scans a directory tree, classifies files by a 5GB threshold, and serves them over TCP port 5001. Files at or above the threshold are transferred individually using kernel zero-copy via `sendfile()`. Files below the threshold are streamed as a tar archive. The pull (client) auto-detects the gateway IP, connects to the host, and receives files using the two-phase protocol. The entire system uses only Python standard library modules.

Q2: Why did you choose a two-phase design instead of a single-phase approach?

Large files benefit enormously from zero-copy `sendfile()`, which avoids copying data through user space. Small files don't benefit as much from zero-copy because the overhead of individual transfers (TCP overhead, round-trip per file) dominates. TAR streaming amortizes this overhead across many small files. The 5GB threshold balances these two approaches based on where the performance curves cross for typical LAN conditions.

Q3: Why a single connection instead of multiple parallel connections?

Simplicity and correctness. A single connection guarantees in-order delivery, eliminates the need for reassembly logic, and avoids TCP congestion control interactions between parallel streams. For a LAN transfer tool where the bottleneck is typically disk I/O or single-link bandwidth, parallel connections wouldn't help and could hurt due to increased overhead. For WAN or very high-bandwidth links, parallel streams could help saturate the pipe.

Q4: Why did you use a custom protocol instead of existing protocols like FTP, SFTP, or HTTP?

Three reasons: (1) Learning: building a custom protocol deepened my understanding of network programming. (2) Control: I have full control over every byte on the wire, enabling optimizations specific to my use case. (3) Dependencies: existing protocols require either external libraries or complex implementations. SFTP requires SSH. FTP uses multiple connections. HTTP adds unnecessary header overhead. My protocol is 11 bytes of header per large file - minimal overhead.

Q5: Walk me through the exact byte format of your protocol header.

Phase 1 header (per large file): [1 byte: flag 0x01] [2 bytes: path length, uint16 big-endian] [N bytes: UTF-8 path] [8 bytes: file size, uint64 big-endian] [N bytes: raw file data]. Phase 2 header: [1 byte: flag 0x02] [streaming tar archive until EOF]. Total Phase 1 overhead per file: 11 bytes + path length. Phase 2 overhead: 1 byte. The struct format is '!HQ' for path_len and filesize, with the flag prepended separately.

Q6: Why big-endian for the binary protocol?

Network byte order is big-endian by convention (RFC 1700). Both Android ARM and Windows x86 are little-endian natively. Using big-endian ensures both platforms interpret the bytes correctly without needing to detect architecture. Python's struct module with '!' prefix handles the conversion transparently.

Q7: How does the client know when one file ends and the next begins?

In Phase 1, the header includes the exact file size (8 bytes). The client reads exactly that many bytes after the header, so it knows precisely where the file ends. The next byte is either another 0x01 flag (more large files) or the 0x02 flag (transition to Phase 2). In Phase 2, the tar format itself delimits file boundaries with headers and padding.

Q8: What happens if the server has no large files? Does Phase 1 get skipped?

Yes. If the `big_files` list is empty after scanning, the server immediately sends the 0x02 flag and starts Phase 2 tar streaming. The client reads the first byte, sees 0x02, and enters Phase 2 receive mode directly. This is handled by the `if big_files:` check before the Phase 1 loop.

Q9: Why did you use `makefile()` for the tar streaming instead of direct socket operations?

Python's `tarfile` module operates on file-like objects. `socket.makefile()` wraps the socket in a buffered file interface that `tarfile` can read from and write to. This allows the `tarfile` module to use its standard streaming API (`mode='w|'` and `'r|'`) without modification. The 8MB buffering parameter matches our socket buffer size for efficient I/O.

Q10: How would you add metadata (file timestamps, permissions) to your protocol?

Option 1: Add fields to the Phase 1 header (`mtime`, `mode` as additional struct fields). Option 2: Use the tar format for all files (tar headers natively contain this metadata). Option 3: Add a separate metadata phase before data transfer. I would probably move to all-tar for metadata since it handles this natively, and use a custom extension only for the size/offset optimization that `sendfile()` provides.

Networking Questions (11-30)

Q11: Why TCP and not UDP?

File transfer requires reliability and in-order delivery. TCP provides both. UDP would require building retransmission, sequencing, congestion control, and flow control at the application layer - essentially reimplementing TCP. The overhead of TCP is negligible compared to the complexity of a reliable UDP protocol for this use case.

Q12: Why port 5001 specifically?

It's in the user/dynamic port range (49152-65535) but memorable. It's not a well-known service port, so unlikely to conflict. In production, I would make it configurable or use a service discovery mechanism.

Q13: Explain why you set SO_REUSEADDR.

After a connection closes, the socket enters TIME_WAIT state for 60 seconds (2*MSL on Linux). Without SO_REUSEADDR, attempting to restart the server within this window fails with "Address already in use." This option allows binding to a port in TIME_WAIT state, which is essential for development and for production services that restart.

Q14: How did you choose 8MB for the socket buffer sizes?

For a typical LAN (1 Gbps, ~1ms RTT), the BDP is about 125KB. The 8MB buffer is 64x the BDP, providing ample headroom for burst handling and kernel scheduling jitter. Testing showed this value provided good throughput without excessive memory usage. For WAN transfers, I would calculate $BDP = \text{bandwidth} * RTT$ and set buffers accordingly.

Q15: Why 1MB write chunks on the client side?

1MB is a sweet spot for disk I/O efficiency - most filesystems and disks operate efficiently at this block size. It's small enough to allow progress display and interruption handling, but large enough to minimize the overhead of Python-level loop iterations. Larger chunks would use more memory; smaller chunks would add loop overhead.

Q16: What happens if the connection drops mid-transfer?

The server catches the exception and closes the socket. The client detects ConnectionResetError or socket.timeout and asks the user whether to retry. If retrying, the client re-establishes the connection. A production version would implement resume: track bytes transferred and restart from the last confirmed position.

Q17: Why a 60-second socket timeout?

The timeout detects stalled transfers. If no data flows for 60 seconds, either the network is down or the peer is unresponsive. This prevents the client from hanging indefinitely. For large files on slow networks, the timeout resets with each byte received, so active transfers don't trigger it.

Q18: How does your system handle the TCP slow start phase for large files?

TCP slow start increases the congestion window exponentially from an initial value (typically 10 MSS on modern Linux). For files much larger than the slow start threshold, this initial phase is negligible. The 8MB socket buffer allows the window to grow large enough to saturate the LAN link. For very large files, TCP spends most of the transfer in congestion avoidance with a large stable window.

Q19: Why does the client read in 1MB chunks instead of reading the whole file at once?

Memory constraints. A file could be 100GB - reading it all into memory would exhaust RAM. Streaming in 1MB chunks bounds memory usage regardless of file size. This is essential for handling files larger than available memory.

Q20: How would you make this work over the internet instead of just LAN?

Challenges: NAT traversal, security, variable bandwidth, higher latency. Solutions: (1) Use a relay server with public IP that both client and server connect to. (2) Add NAT traversal (STUN/TURN/ICE). (3) Implement TLS for encryption. (4) Add adaptive buffer sizing based on measured RTT. (5) Implement chunked transfer with resume. (6) Add compression for text files.

Q21: What would tcpdump show for your protocol?

tcpdump -i any port 5001 -X would show: TCP three-way handshake, then the 0x01 flag byte followed by the binary header (path length, path bytes, file size in big-endian), then the raw file data. For Phase 2, the 0x02 flag followed by tar archive headers (512 bytes each with filename, mode, uid, size, etc.) and file content padded to 512-byte boundaries.

Q22: How does your gateway auto-detection work?

Two-tier approach: Primary method uses PowerShell Get-NetRoute to find the default route's next hop with a 3-second timeout. If PowerShell fails, fallback parses ipconfig output, prioritizing the Wi-Fi adapter section. All candidates are validated against an IPv4 regex. This handles different Windows versions and configurations.

Q23: What if the network has multiple gateways?

The current implementation returns the first valid gateway. For multiple gateways, I would present all discovered gateways to the user for selection, or implement a reachability test (ping each gateway and choose the responsive one). In a corporate environment, I'd use DHCP options or a configuration file.

Q24: Why does the server listen on 0.0.0.0 instead of a specific IP?

0.0.0.0 means "all available network interfaces." The server might have multiple IPs (Wi-Fi, Ethernet, loopback). Binding to 0.0.0.0 allows connections from any interface, which is appropriate for a file transfer tool. Binding to a specific IP would restrict access to that interface only.

Q25: How would you add IPv6 support?

Change socket family from `AF_INET` to `AF_INET6`. The binary protocol remains the same. Use `socket.getaddrinfo()` for address resolution that works with both IPv4 and IPv6. Bind to `:::` for dual-stack. The struct packing and protocol logic don't change - only the addressing layer.

Q26: What is the maximum file size your protocol can handle?

The file size field is 64 bits (`uint64`), so the theoretical maximum is $2^{64} - 1$ bytes (~18 exabytes). Practical limits come from the filesystem (ext4 max file size is 16TB, NTFS is 16TB), available disk space, and the time required to transfer. The 5GB threshold is configurable.

Q27: How does TCP keepalive interact with your timeout?

I don't use TCP keepalive; I use Python's `socket.settimeout(60)`. TCP keepalive defaults to 2 hours of inactivity before sending probes, which is too long for detecting stalled transfers. My 60-second application-level timeout is more aggressive and appropriate for interactive file transfers.

Q28: What happens when the server sends data faster than the client can write to disk?

TCP flow control handles this automatically. As the client's receive buffer fills, it advertises a smaller window. The server respects this and stops sending. If the buffer completely fills, the server blocks on `sendfile()` until space opens up. This backpressure propagates all the way to the disk write speed.

Q29: How would you measure the actual throughput of your transfer?

Time the transfer from first byte sent to last byte received: `throughput = total_bytes / elapsed_time`. I already capture start time with `time.time()`. For more precision, use `time.perf_counter()`. For ongoing measurement, track bytes transferred over time windows (every second) to see throughput variation.

Q30: Why not use Python's built-in `http.server` for file transfer?

`http.server` is request-response based, designed for browsing, not bulk transfer. It lacks: zero-copy optimization, streaming tar packaging, progress tracking, connection recovery, and custom protocol features. It's also not optimized for large file performance - each request requires HTTP header parsing and response generation.

Operating Systems Questions (31-45)

Q31: Explain exactly what happens inside the kernel when you call `sendfile()`.

`sendfile()` is a system call that transfers data between two file descriptors without copying through user space. Internally: (1) The kernel checks fd permissions. (2) It locates the file's pages in the page cache. (3) It passes page references through a pipe buffer using `splice()`. (4) The socket layer transmits pages via DMA to the NIC. (5) The kernel adjusts file offset and returns bytes transferred. No data ever enters the process's address space.

Q32: What is zero-copy and how many copies does your system eliminate?

Zero-copy eliminates CPU-mediated data copies between kernel and user space. Traditional path: disk → kernel pagecache (DMA copy 1) → user buffer (CPU copy 2) → kernel socket buffer (CPU copy 3) → NIC (DMA copy 4). With `sendfile()`: disk → kernel pagecache (DMA copy 1) → kernel passes pages by reference → NIC (DMA copy 2). Eliminates 2 CPU copies and 2 context switches.

Q33: Why is `sendfile()` faster than `read()` + `write()` for large files?

Three reasons: (1) Eliminates 2 memory copies through user space, reducing CPU usage and memory bandwidth consumption. (2) Eliminates 2 context switches (`read` → `return` → `write` → `return` become single `sendfile` syscall). (3) Enables kernel optimizations like DMA gather-scatter where the NIC reads directly from page cache pages.

Q34: What are the limitations of `sendfile()`?

(1) Cannot transform data in transit - no encryption, compression, or modification. (2) Source must be a file supporting `mmap()`; destination must be a socket. (3) Transfer must start from a file offset (can't send from memory buffer). (4) Error handling is less granular - you can't inspect data being sent. (5) Platform dependent - not all OSes implement it equally.

Q35: What system calls does your application make during a transfer?

Host: `socket()`, `setsockopt()`, `bind()`, `listen()`, `accept()`, `open()`, `sendfile()`, `write()` (for headers), `close()`. Pull: `socket()`, `setsockopt()`, `connect()`, `read()` (via `makefile`), `write()` (to disk), `close()`. Can verify with `strace -e trace=network, file python script.py`.

Q36: Why is your 8MB buffer size significant in terms of the kernel?

The socket buffer is allocated in kernel memory. 8MB per socket means memory usage scales with concurrent connections. For single-connection transfers this is fine, but for 1000 concurrent connections, that's 8GB of kernel memory just for buffers. The buffer also affects TCP window size advertisement - larger buffers allow larger windows, which help high-BDP paths.

Q37: How does the kernel handle the socket file descriptor?

A socket fd is an index into the process's file descriptor table, pointing to a file structure in the kernel. The file structure points to a socket structure containing: protocol state (TCP state machine), receive/send buffers, connection info (local/remote address), options, and callback functions for the protocol family.

Q38: What happens to your socket if the process crashes?

The kernel closes all file descriptors when a process exits or crashes. For TCP sockets, this sends a FIN to the peer, initiating the four-way termination. If the crash is abrupt (SIGKILL), the kernel may send RST instead. Data in the socket buffers that hasn't been sent is lost. The socket enters TIME_WAIT if the local end initiated close.

Q39: How does the page cache help your file transfers?

When a file is read, the kernel caches pages in RAM (page cache). Subsequent reads or sendfile() calls use these cached pages instead of disk I/O. For repeated transfers of the same file, the second transfer is much faster (RAM speed vs disk speed). The kernel manages eviction using LRU-like algorithms when memory pressure occurs.

Q40: Explain the difference between blocking and non-blocking I/O in the context of your project.

Your project uses blocking I/O: socket operations block until data is available or space is free. This simplifies the code - sequential reads and writes without event loops. Non-blocking I/O would require select/poll/epoll to know when sockets are ready, adding complexity but enabling single-threaded handling of multiple connections. For a simple transfer tool, blocking I/O is the right choice.

Q41: What scheduling priority does your Python process have?

By default, normal scheduling (SCHED_OTHER) with nice value 0. This means the CFS scheduler gives it a fair share of CPU time. For time-critical transfers, we could increase priority with nice -n -10 (requires root) or use SCHED_FIFO for real-time scheduling. However, since the transfer is I/O-bound, CPU priority rarely matters - the process spends most of its time waiting for I/O.

Q42: How much memory does your transfer process use?

The Python process memory is bounded: socket buffers (8MB send + 8MB receive in kernel, not counted in process RSS), 1MB user-space write buffer, Python interpreter overhead (~20-50MB), and protocol metadata. The key design decision is streaming - memory usage is independent of file size because we never load entire files into memory.

Q43: What file descriptors are open during a transfer?

Host: stdin(0), stdout(1), stderr(2), server socket fd, client connection fd, file fd for each large file being sent. Pull: stdin, stdout, stderr, connection socket fd, output file fd. View with `ls -la /proc/PID/fd/` or `lsof -p PID`.

Q44: How does your system handle signals?

KeyboardInterrupt (SIGINT, Ctrl+C) is caught by Python and raises KeyboardInterrupt exception. Both scripts catch this in try/except blocks for graceful cleanup. SIGTERM would also be caught by Python's default handler. No custom signal handlers are installed - the finally blocks ensure socket closure on exit.

Q45: What is DMA and how does it relate to your project?

DMA (Direct Memory Access) allows hardware devices to transfer data to/from memory without CPU involvement. In your project, DMA is used twice: (1) Disk controller reads file data into kernel page cache. (2) NIC sends data from socket buffers over the network. `sendfile()` leverages DMA by keeping data in kernel space, allowing the NIC to read directly from page cache pages in some configurations (scatter-gather).

Performance Questions (46-60)

Q46: What is the bottleneck in your file transfer system?

For large files on LAN: typically disk read speed on the host or disk write speed on the client. 1 Gbps Ethernet can sustain ~125 MB/s, which SATA SSDs can match but HDDs may not. For many small files: the per-file overhead of tar headers and filesystem operations becomes significant. CPU is rarely the bottleneck due to zero-copy.

Q47: How would you measure if the bottleneck is network or disk?

(1) Run `iperf3` between the same hosts - if `iperf3` saturates the link, network is not the bottleneck. (2) Monitor disk I/O with `iostat` during transfer - if disk utilization is near 100%, disk is the bottleneck. (3) If neither is saturated, check CPU with `top`. (4) Check for network retransmissions with `netstat -s`.

Q48: How would you make the transfer 10x faster?

(1) Parallel transfers: multiple connections or pipelining. (2) Compress data before transfer (for compressible content). (3) Use faster network (10 Gbps). (4) Use faster storage (NVMe SSD). (5) Increase parallelism with async I/O. (6) Enable jumbo frames (MTU 9000) to reduce per-packet overhead. (7) Tune TCP: increase initial congestion window, enable BBR congestion control.

Q49: How much overhead does your protocol header add?

Phase 1: 11 bytes fixed + path length per file. For a 5GB file with a 50-character path, overhead is 61 bytes out of 5GB = 0.0000012%. Phase 2: 1 byte flag + tar padding (512-byte blocks). For small files, tar overhead can be significant (a 100-byte file uses 512 bytes of header + 512 bytes of data = 1024 bytes, 10x expansion).

Q50: Why is tar padding to 512 bytes wasteful for small files?

The tar format was designed for tape drives that read/write 512-byte blocks. A 1-byte file uses 512 bytes of header + 512 bytes of data block = 1024 bytes. For directories of many tiny files, this overhead adds up. The 5GB threshold helps by bundling small files where per-file overhead dominates. Alternative: a custom streaming format without block alignment would be more space-efficient.

Q51: How would you profile your Python code?

Use `cProfile` for function-level profiling: `python -m cProfile -s cumulative script.py`. For line-by-line: `line_profiler` with `@profile` decorator. For memory: `memory_profiler`. For specific functions: `timeit` module. Focus on hot paths - for UnMTP, the interesting parts are the loop that reads and writes chunks, and the tar streaming.

Q52: What would happen if you increased the 5GB threshold to 50GB?

Fewer files would use `sendfile()`, more would go through tar streaming. If most files are small, this might not hurt. But for a 10GB file now going through tar, it would be read into the tar streaming buffer in user space instead of zero-copy kernel transfer, potentially reducing throughput and increasing CPU usage. The optimal threshold depends on the file size distribution.

Q53: What would happen if you decreased the 5GB threshold to 5MB?

Almost all files would use `sendfile()` individually. Each file would get its own 11-byte header and separate kernel transfer. For directories with thousands of small files, the per-file overhead (TCP ACKs, context switches, header bytes) would dominate, potentially reducing throughput compared to tar bundling. The 5MB threshold would likely hurt performance for typical file distributions.

Q54: How does CPU cache affect your transfer performance?

For `sendfile()`-based transfers, CPU cache has minimal impact because data doesn't touch user space. For the Phase 2 tar streaming and Phase 1 client write loop, the 1MB buffer likely fits in L3 cache, allowing efficient copying. Sequential access patterns (reading/writing large blocks) have good cache and prefetch behavior. Random small file access would hurt cache efficiency.

Q55: How much CPU does your transfer use?

Due to `sendfile()`, large file transfers should use minimal CPU (< 5% of one core). The client does more work: reading from socket and writing to disk in a Python loop, which uses more CPU. For Phase 2 tar streaming, both sides use Python-level I/O, so CPU usage is higher but still modest for LAN speeds.

Q56: What is the theoretical maximum throughput of your system?

On a 1 Gbps LAN with jumbo frames: ~940 Mbps usable (Ethernet overhead). With `sendfile()` and fast disks, can approach this. Without zero-copy, CPU and memory bandwidth become limiting factors. On 10 Gbps networks, single TCP connection may not saturate the link due to single-stream TCP limitations - parallel connections or RDMA would help.

Q57: How does your system handle bursty network conditions?

The 8MB socket buffers absorb short bursts. TCP's congestion control smooths out transmission rate. If the burst exceeds buffer capacity, packets are dropped and TCP retransmits. The 60-second timeout handles sustained stalls. For very bursty conditions (microbursts common in trading), larger buffers or application-level flow control would help.

Q58: How would you add compression and what would be the tradeoff?

Options: (1) Pre-compress files before transfer (best for known content). (2) Stream compression with `zlib/lz4` (adds CPU overhead, reduces bytes transferred). (3) Compress tar archive on-the-fly. Tradeoff: CPU usage vs network bandwidth. For LAN transfers, compression rarely helps because network is faster than CPU. For WAN, compression can significantly help, especially for text.

Q59: How would you add parallel transfers?

(1) Multiple TCP connections, each handling a subset of files. (2) Single connection with request multiplexing (like HTTP/2). (3) Split large files into chunks transferred in parallel. Implementation: use threading or asyncio on both sides, add a file allocation strategy (round-robin or by size), and ensure the receiver reassembles correctly. Each connection needs its own port or stream ID.

Q60: How would you optimize for transferring millions of tiny files?

(1) Reduce per-file overhead: pack multiple files per tar entry or use a custom binary format. (2) Eliminate filesystem metadata operations: batch create directories. (3) Use async I/O to overlap network and disk operations. (4) Consider a packed archive format that avoids individual file creation overhead. (5) Parallelize extraction across multiple threads.

Security Questions (61-70)

Q61: What are the security vulnerabilities in your system?

(1) No encryption - data transmitted in plaintext over the network. (2) No authentication - any device on the network can connect and receive files. (3) No integrity verification - no checksums to detect tampering or corruption. (4) Path traversal risk - `tar.extractall()` without path validation could write outside the destination directory. (5) No rate limiting - client can request unlimited data. (6) Information disclosure - directory structure is visible to any connected client.

Q62: How would you add TLS encryption to your protocol?

Use Python's `ssl` module: `wrap_socket()` around the TCP socket. Server loads a certificate and private key; client validates the certificate chain. For mutual authentication, the client also presents a certificate. After handshake, all data is encrypted. Tradeoff: CPU overhead for encryption (mitigated by AES-NI hardware instructions) and increased latency for the handshake.

Q63: How would you authenticate clients?

Options: (1) Pre-shared key or token sent in protocol header after connection. (2) TLS client certificates (strongest). (3) Token-based: server generates token, client must present valid token. (4) IP allowlist for LAN environments. I would implement TLS with client certificates for production - it provides both encryption and authentication without custom auth code.

Q64: How would you verify file integrity?

Add a SHA-256 or BLAKE3 hash for each file. Options: (1) Hash sent in header, verified after write. (2) End-of-transfer summary with all hashes. (3) Per-chunk CRC for streaming verification. For large files, compute hash while writing (incremental hash update) and compare at the end.

Q65: What is the tar path traversal vulnerability and how does it affect your system?

A malicious tar archive can contain entries with paths like `"../..../etc/passwd"` that escape the intended extraction directory. Python 3.12+ warns about this. Mitigation: validate extracted paths using `os.path.commonpath()` to ensure the resolved path is within the destination directory. Use `tarfile.extractall(members=validated_members)` or set `filter='data'` in Python 3.12+.

Q66: How would you prevent directory traversal attacks?

For each extracted path: `resolved = os.path.abspath(os.path.join(dest_dir, entry.name))`; then check `os.path.commonpath([dest_dir, resolved]) == dest_dir`. Reject any entry where this check fails. Also reject entries containing `'..'` components. This ensures all writes are within the destination directory.

Q67: How would you add access control to limit which files can be transferred?

(1) Configuration file specifying allowed source directories. (2) File pattern allowlists/denylists. (3) Per-user access policies. (4) Audit logging of all transfers. The server should validate requested paths against allowed directories before any file operation.

Q68: What logging would you add for security auditing?

(1) Connection events: client IP, timestamp, authentication result. (2) Transfer events: files transferred, sizes, duration, checksums. (3) Error events: failed connections, timeouts, protocol errors. (4) Use structured logging (JSON) for automated analysis. (5) Log to both file and centralized logging system. (6) Protect logs from tampering (append-only, remote shipping).

Q69: How would you rate limit transfers?

(1) Per-IP connection limits using a connection counter. (2) Per-client bandwidth caps using token bucket algorithm. (3) Global connection pool to prevent resource exhaustion. (4) Exponential backoff on repeated connection attempts. Token bucket is ideal: client gets tokens at a fixed rate, each byte consumes a token, excess requests queue or drop.

Q70: How would you protect against a malicious client that connects but never reads data?

TCP's built-in mechanisms help: the send buffer fills up, `sendfile()` blocks, and eventually the connection stalls. Application-level protection: (1) Set `SO_SNDTIMEO` for send timeouts. (2) Monitor transfer progress - if no bytes acknowledged within a timeout, close connection. (3) Limit maximum transfer time per connection. (4) Use non-blocking I/O with `select()` to detect stuck peers.

Python Deep-Dive Questions (71-80)

Q71: Why did you use struct instead of JSON or protobuf for serialization?

(1) Size: struct header is 11 bytes vs hundreds for JSON. (2) Speed: binary packing/unpacking is much faster than JSON parsing. (3) No dependencies: struct is standard library. Protobuf would require an external package. (4) Simplicity: for a fixed protocol with few fields, struct is simpler than defining a schema. (5) Predictability: fixed-size fields enable efficient parsing without scanning.

Q72: How does Python's socket.sendfile() differ from the OS sendfile()?

Python's `socket.sendfile()` is a high-level wrapper. It first tries the OS `sendfile()` syscall. If the OS doesn't support it (e.g., Windows), it falls back to a read-send loop in Python. It also handles the file offset tracking and supports non-blocking sockets. The return value is bytes sent, matching the OS behavior when available.

Q73: Why did you use os.walk() instead of pathlib.Path.rglob()?

`os.walk()` provides directory-top-down traversal with immediate directory/file separation. It yields `(dirpath, dirnames, filenames)` tuples, which is efficient for our use case of scanning and classifying files. `pathlib.Path.rglob('*')` is more modern and readable but returns `Path` objects with overhead. For a performance-sensitive scanner, `os.walk()` is slightly more efficient.

Q74: What is the difference between bytes and bytearray, and which should you use where?

`bytes` is immutable - like a frozen sequence of integers 0-255. `bytearray` is mutable - supports item assignment and methods like `extend()`. Use `bytes` for fixed data (protocol constants, received data you don't modify). Use `bytearray` when building buffers incrementally or modifying data in place. For UnMTP's fixed headers, `bytes` is appropriate.

Q75: How would you use memoryview to optimize your transfer?

memoryview provides zero-copy slicing of byte buffers. Instead of copying slices with `buffer[start:end]`, a memoryview references the same memory. For the 1MB write chunks on the client, using memoryview could avoid buffer copying if we were doing transformations. In the current design, the benefit is limited because we read into a buffer and write it directly.

Q76: What is the purpose of the finally block in your socket code?

The finally block ensures the socket is closed regardless of whether the transfer completed successfully or raised an exception. Without this, an error during transfer would leave the socket open, leaking the file descriptor and potentially keeping the port in use (without `SO_REUSEADDR`). This is critical resource management for long-running processes.

Q77: How would you use context managers to improve your code?

Wrap socket creation and file opening in context managers (with statement). This automatically handles cleanup even on exceptions. Python's `socket.create_connection()` supports context protocol. Custom context managers using `@contextmanager` could wrap the two-phase transfer logic, ensuring proper cleanup of both socket and file resources.

Q78: Why is your protocol designed as stateless across transfers?

Each transfer is independent: connect, transfer all files, disconnect. There's no session state maintained between connections. This simplifies recovery (just reconnect and restart) and server implementation (no session tracking). The tradeoff is that resume requires re-transferring all files rather than continuing from an offset.

Q79: How would you implement resumable transfers?

Add a RESUME command to the protocol where the client sends a list of files it already has (with checksums or sizes). The server only sends missing or changed files. For interrupted large files, track offset and resume from that position using `sendfile()` with offset parameter. Add a manifest phase before data transfer.

Q80: How does Python's garbage collection affect long-running transfers?

CPython uses reference counting for immediate cleanup and generational GC for cycles. For UnMTP, objects are short-lived (buffers, file objects) and reference counted away immediately. The generational GC runs rarely and shouldn't interfere. For multi-hour transfers, calling `gc.collect()` between phases could prevent gradual memory growth from rare cycles.

Debugging Questions (81-90)

Q81: A user reports "Connection refused." What is your debugging process?

1) Verify the server process is running (`ps aux | grep host`). 2) Check if port 5001 is listening (`ss -tlnp | grep 5001`). 3) Try connecting locally on the server (`nc -vz localhost 5001`). 4) Check firewall rules (`iptables -L -n`, `ufw status`). 5) Verify the client is using the correct IP and port. 6) Check if another process is using the port. 7) Use `tcpdump` to see if SYN packets arrive at the server.

Q82: Transfer starts fast then slows to a crawl. How do you debug?

1) Check CPU (`top`) - is Python consuming all CPU? 2) Check memory (`free -h`) - swapping? 3) Check disk I/O (`iostat -x 1`) - disk saturated? 4) Check network (`nload`, `iftop`) - bandwidth drop? 5) Check TCP retransmissions (`netstat -s | grep retrans`). 6) Check socket buffer state (`ss -tin`). 7) Profile Python (`cProfile`). 8) Check for thermal throttling (`dmesg | grep thermal`).

Q83: The client hangs after receiving exactly 2GB of data. Why?

Classic 32-bit integer overflow. If the client uses a 32-bit signed integer for tracking received bytes, it overflows at 2^{31} (2GB). Check: (1) Are you using Python `int` (unlimited) or a C extension with 32-bit `int`? (2) Is the filesystem FAT32 (max 4GB file)? (3) Is there a 2GB limit in the tarfile module for certain Python versions? Solution: ensure 64-bit integers throughout.

Q84: How would you use `strace` to debug your file transfer?

`strace -e trace=network,file,desc -o trace.log python host.py` shows all socket, file, and descriptor syscalls. Look for: unexpected `close()` calls, short reads/writes, repeated reads returning `EAGAIN`, `sendfile()` returning errors. For the client: `strace -e trace=read,write,recvfrom,sendto` shows actual data flow timing.

Q85: The tar extraction fails with "truncated header." What happened?

The tar stream was cut off before completion. Causes: (1) Network disconnection mid-transfer. (2) Server crash. (3) Client timeout. (4) Incorrect file size in protocol causing client to read wrong amount. (5) Server sending wrong flag sequence. Check connection logs, verify protocol flag sequence, and add chunked transfer with integrity checks.

Q86: How would you add structured logging to your system?

Use Python's logging module with JSON formatter. Log: connection events, file start/completion, errors, performance metrics. Use different log levels (INFO for transfers, DEBUG for protocol details, ERROR for failures). Add correlation IDs to track transfers across client and server logs. Rotate logs with RotatingFileHandler.

Q87: How would you monitor your transfer system in production?

Metrics to collect: transfer rate (MB/s), latency, success/failure rate, active connections, bytes transferred. Tools: Prometheus for metrics collection, Grafana for visualization. Instrument code with counters and histograms. Alert on: error rate spikes, throughput drops, connection failures. Add health check endpoint.

Q88: A file transfers but has different content on the client. How do you find the bug?

1) Compare checksums (sha256sum) on both sides. 2) Check if the size matches (ls -l). 3) Use cmp or diff to find where bytes differ. 4) Add logging of bytes sent/received at offsets. 5) Check for text mode vs binary mode issues (must use 'rb'/'wb'). 6) Verify struct pack/unpack format strings match. 7) Check for buffer reuse bugs.

Q89: The server shows high CPU usage during transfer. Why?

sendfile() should use near-zero CPU. High CPU suggests: (1) Fallback to read-send loop (OS doesn't support sendfile() for this combination). (2) Excessive logging or console output. (3) Python-level processing of data instead of kernel transfer. (4) Very small buffer sizes causing frequent syscalls. (5) Check with strace if sendfile syscall is actually being used.

Q90: How would you reproduce a race condition in your transfer?

For UnMTP with single connection, race conditions are minimal. To test: (1) Use multiple simultaneous clients (if you add concurrency). (2) Introduce network delays (tc qdisc add dev eth0 root netem delay 100ms). (3) Simulate packet loss (tc qdisc add dev eth0 root netem loss 1%). (4) Kill processes at random points to test recovery. (5) Use stress testing tools to load the system.

Scalability and Design Questions (91-100)

Q91: How would you scale this to handle 1000 concurrent transfers?

(1) Use `asyncio` or `epoll` for single-threaded concurrency (10,000+ connections). (2) Add worker pool for disk I/O. (3) Use shared nothing architecture - each transfer is independent. (4) Load balance across multiple server instances. (5) Use distributed storage backend (S3). (6) Add connection pooling and backpressure. (7) Monitor and auto-scale based on load.

Q92: How would you design this for a data center with 10,000 servers?

Centralized metadata service (file catalog, permissions), distributed transfer agents on each server, message queue for transfer coordination (Kafka/RabbitMQ), object storage backend (Ceph/S3), monitoring with Prometheus/Grafana, service discovery (Consul/etcd), TLS everywhere, rate limiting at edge, CDN for hot files.

Q93: How would you add a progress bar to your transfer?

Track: total bytes to transfer (from scan), bytes transferred so far, current file. Update progress bar each iteration. Libraries: `tqdm` for simple, `rich.progress` for fancy. For network display, send progress updates through a separate channel or multiplex into the protocol. Calculate ETA: $(\text{elapsed} / \text{percent_complete}) - \text{elapsed}$.

Q94: How would you make your system work on macOS?

macOS supports `sendfile()` but with different semantics. The `tarfile` and `socket` modules work the same. Main changes: gateway detection (use `netstat -rn | grep default` or `route get default`), path handling (similar to Linux), socket options (`SO_REUSEADDR` works, some Linux-specific options don't). The protocol itself is platform-independent.

Q95: How would you add checksums without changing the protocol much?

Option 1: Compute file hash before transfer, send as trailing metadata after the file data. Option 2: Add a checksum field to the existing header (requires version negotiation). Option 3: Send a separate "checksum phase" after all files. Option 4: Use a self-describing format where the last 32 bytes of each file transfer are the SHA-256 hash (client strips and verifies).

Q96: How would you handle symbolic links during transfer?

Options: (1) Follow symlinks and transfer the target file (current tar default). (2) Preserve symlinks in the tar archive (dereference=False). (3) Skip symlinks entirely. (4) Error on symlinks for security. The right choice depends on requirements. Following symlinks is simplest but may cause duplicates or cycles.

Q97: How would you implement delta sync (only transfer changed parts)?

Use rsync algorithm: divide file into fixed-size blocks, compute rolling hash of each block. Client sends hashes to server. Server sends only blocks that differ. For large files with small changes, this is dramatically more efficient. Implementations: librsync, or custom implementation using Adler32 rolling hash + MD5.

Q98: How would you handle files that are modified during transfer?

(1) Detect modification: stat before and after transfer, compare mtime/size. (2) Lock files during transfer (flock on Linux) - prevents writes. (3) Snapshot files first (copy to temp, transfer snapshot). (4) Compute checksum before and after, retry if changed. (5) Log warning and skip files that changed during transfer.

Q99: How would you write tests for your file transfer system?

Unit tests: test struct pack/unpack, test file classification logic, test path validation. Integration tests: start server, transfer known files, verify contents match. Edge cases: empty files, 0-byte directories, files with special characters, very large files, network interruption. Use pytest with fixtures for temp directories. Use loopback interface (127.0.0.1) for testing.

Q100: If you had one week to improve this project, what would you do?

Priority order: (1) Add TLS encryption for security. (2) Add integrity verification (SHA-256 checksums). (3) Implement resume capability for interrupted transfers. (4) Add proper logging and monitoring. (5) Write comprehensive tests. (6) Add progress reporting. (7) Add rate limiting and bandwidth throttling. (8) Refactor into classes with clean interfaces. (9) Add configuration file support. (10) Write documentation.

Tradeoff and Deep-Dive Questions (101-115)

Q101: Tradeoff: simplicity vs performance in your design.

I chose simplicity: single connection, blocking I/O, procedural code. This makes the system reliable and easy to understand. Performance comes from the zero-copy `sendfile()` for large files and tar streaming for small files. More performance could be gained with parallel connections and async I/O, but at significant complexity cost. For a LAN transfer tool, the current design achieves near-disk-speed which is sufficient.

Q102: Why not use `mmap()` + `write()` instead of `sendfile()`?

`mmap()` maps a file into virtual address space; you still need `write()` to copy it to the socket, which involves a kernel copy from page cache to socket buffer. `sendfile()` avoids even that copy by passing page references internally. `mmap()` is useful when you need to inspect or modify data before sending; `sendfile()` is faster for direct file-to-socket transfer.

Q103: Why is your tar streaming approach better than ZIP for this use case?

tar supports true streaming (`mode='w|'`) - write sequentially without seeking. ZIP requires seeking to write the central directory at the end. For a socket stream where seeking is impossible, tar is the natural choice. ZIP would require buffering the entire archive in memory or on disk to write the directory, defeating the streaming design.

Q104: Why did you choose a fixed 5GB threshold over dynamic adaptation?

Simplicity and predictability. A fixed threshold is easy to understand and configure. Dynamic adaptation would require runtime measurement of transfer rates for both methods, adding complexity and warm-up time. For known use cases (LAN transfer of mixed media files), a fixed threshold works well. Dynamic adaptation would make sense for unknown or varying network conditions.

Q105: How does your system handle the case where a file is exactly 5GB?

The condition is `size ≥ BIG_FILE_THRESHOLD`, so a 5GB file uses `sendfile()`. This is intentional: files at the threshold should use the optimized path. The exact boundary is less important than consistency - both sides use the same logic (server classifies, client reads flags), so they agree on the phase for each file.

Q106: Why is there no versioning in your protocol?

For a v1 tool between known endpoints, protocol versioning was deferred. Adding it later: reserve the first byte's high bits for version (e.g., 0x81 = v1, Phase 1), or add a version handshake at connection start. Versioning is essential for backward compatibility when evolving the protocol.

Q107: How does your design change if latency increases from 1ms to 200ms?

Higher latency increases TCP round-trip time, slowing down: (1) Connection establishment (3 RTT). (2) Slow start phase. (3) ACK clocking. Solutions: increase socket buffers to match higher BDP ($\text{BDP} = 1\text{Gbps} * 200\text{ms} = 25\text{MB}$), enable TCP window scaling, use initial congestion window of 10+, consider parallel TCP streams or QUIC.

Q108: Why is your protocol not idempotent?

Each transfer starts fresh with no knowledge of previous attempts. Making it idempotent would require: client tracking what it received, server skipping already-sent files, checksum comparison. This would add a "manifest phase" before data transfer. The current design prioritizes simplicity over idempotency - appropriate for a manual transfer tool, not for automated sync.

Q109: How would your design differ for SSD vs HDD source storage?

SSDs have much better random I/O performance, so parallel reads would help. HDDs suffer from seek time, so sequential access (current design with `os.walk`) is optimal. For HDD: larger read buffers, avoid concurrent disk access, maintain sequential read patterns. For SSD: could use parallel `sendfile()` calls from different files.

Q110: What would you change if you were transferring to 100 clients simultaneously?

Current design is 1-to-1. For 1-to-many: (1) Use multicast UDP for small files. (2) Use a CDN-like cache layer. (3) Implement peer-to-peer distribution (BitTorrent protocol). (4) Pre-stage files on edge servers. (5) For large files to many clients, parallel unicast with `sendfile()` would saturate server network - need distribution architecture.

Q111: Why did you use subprocess for gateway detection instead of a Python library?

There is no cross-platform Python standard library function for gateway detection. `subprocess` calls the native OS tools (PowerShell, `ipconfig`) which are authoritative. Adding a dependency just for gateway detection would violate the zero-dependency design goal. A pure Python implementation would need to parse routing tables, which is platform-specific anyway.

Q112: How does your retry logic work and what are its limitations?

The client has a `while True` loop around connection attempts. `ConnectionRefusedError` resets the IP and re-prompts. `ConnectionResetError` asks the user whether to retry. `KeyboardInterrupt` exits cleanly. Limitations: no automatic retry limit, no exponential backoff, no circuit breaker pattern, resume not implemented. Production would add: max retries, backoff, and resume from offset.

Q113: What is the memory footprint of your server during a transfer?

Minimal and constant regardless of file size. Key components: Python interpreter (~30MB), socket buffers (8MB in kernel space, not process RSS), 1MB protocol buffer for header generation, file descriptor for current file. No file content is loaded into Python memory due to `sendfile()`. For Phase 2 tar streaming, the `tarfile` module has a small internal buffer.

Q114: How does your system handle filesystem encoding differences between platforms?

Paths are sent as UTF-8 bytes, which both Android (Linux) and Windows can decode. Python's `os.fsencode/fsdecode` handle platform-specific encoding. On Windows, paths may use different path separators (backslash vs forward slash) - `os.path.normpath` handles this. The protocol doesn't encode paths with null bytes, which would cause issues.

Q115: What did you learn from this project that surprised you?

"I was surprised by how much performance difference zero-copy makes - it wasn't just theoretical. Watching CPU drop from 30% to near-zero with `sendfile()` was eye-opening. I also learned that protocol design involves many subtle decisions: endianness, header size, flag design. And I gained appreciation for how much TCP handles for us - reliability, flow control, congestion management - all happening transparently."

Revision Checklist - Project Discussion

- [] Can explain the two-phase architecture and rationale in 2 minutes
- [] Can draw the protocol header format from memory
- [] Can explain `sendfile()` and zero-copy with kernel-level detail
- [] Can defend every design decision with tradeoffs
- [] Can list 5+ security vulnerabilities and mitigations
- [] Can describe how to scale to 1000+ concurrent transfers
- [] Can debug common network issues systematically
- [] Can explain TCP internals relevant to the project

- [] Can discuss what you'd improve with more time
- [] Can connect project decisions to OS and networking concepts
- [] Have written answers to all 115 questions above
- [] Have practiced speaking answers aloud at least once

END

Final Notes

Quick Reference: 8-Week Schedule Summary

Table Final-1 Week-by-Week Focus

Week	Primary	Hours	Key Deliverable
1	Computer Networks - Layers 1-3	40	Wireshark labs complete
2	Computer Networks - TCP/UDP + Tools	40	Custom socket programs
3	Operating Systems - Core	40	Process/thread exercises
4	Operating Systems - Memory + I/O	40	Zero-copy experiments
5	Python Systems + Concurrency	40	Mini-tools built
6	DSA Intensive + File Systems	40	50 DSA problems solved
7	System Design + Supporting Topics	40	Mock interview #1
8	Project Discussion + Behavioral	40	All 115 questions answered

Day-Before-Interview Checklist

- [] Review all 115 project discussion answers
- [] Re-read TCP three-way handshake and congestion control
- [] Practice drawing the protocol header from memory
- [] Review `sendfile()` internals and zero-copy concepts
- [] Skim through OS concepts: virtual memory, scheduling, synchronization
- [] Review 5 DSA problems (array, tree, graph, heap, binary search)
- [] Prepare questions to ask the interviewer about the team and role

- [] Sleep 8 hours
- [] Arrive 15 minutes early (virtual: test audio/video)
- [] Bring notebook for diagrams

Key Reminders

WHAT D. E. SHAW VALUES IN THIS ROLE

- **Systems thinking:** Understanding the full stack from application to hardware
- **Practical debugging:** Methodical approach to production issues
- **Performance awareness:** Knowing where bottlenecks live and how to measure
- **Network fundamentals:** TCP deep knowledge is non-negotiable
- **Linux comfort:** Command line fluency and OS internals
- **Communication:** Explaining technical decisions clearly

Recommended Final Review Order

1. Chapter 18: Project Discussion (most important - 50% of your preparation time)
2. Chapter 1: Computer Networks - Transport Layer section
3. Chapter 2: Operating Systems - I/O and Zero-Copy section
4. Chapter 3: Python - socket and struct modules
5. Chapter 5: Debugging methodology
6. Chapter 16: Behavioral questions

* * *

This handbook was prepared specifically for the D. E. Shaw Front Office Technical Associate role.

Focus on depth over breadth. Your project is your superpower - own it.

Good luck.